Parcial II

2025

Camila Tabone
44.450.053

Universidad Atlántida
Laboratorio de Datos

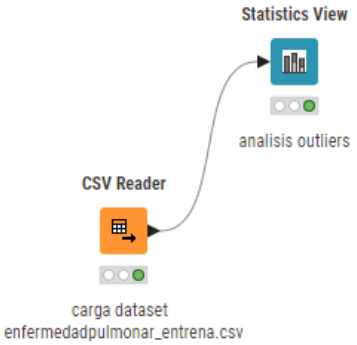
Índice

Índice....1
Preprocesamiento dataset Enfermedad Pulmonar....2
Preprocesamiento dataset de PC....7
Árbol....20
SVM....25
Random Forest....29
KNN....32
K-Means....34
Clustering Jerárquico....37
DBScan....39
Conclusiones obtenidas de las actividades de agrupamiento....40
Flujo final de datos....41

Preprocesamiento dataset Enfermedad Pulmonar

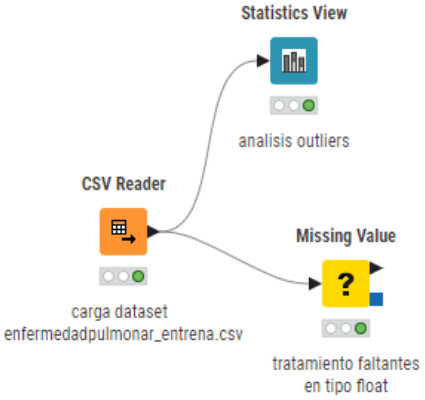
Trabajaremos con el dataset enfermedadpulmonar.csv que contiene datos médicos sobre cuatro tipos de enfermedades pulmonares. Nuestro objetivo es predecir el diagnóstico en función de los datos relevados.
Realizamos la carga del dataset enfermedadpulmonar_entrena.csv mediante el nodo CSV Reader .

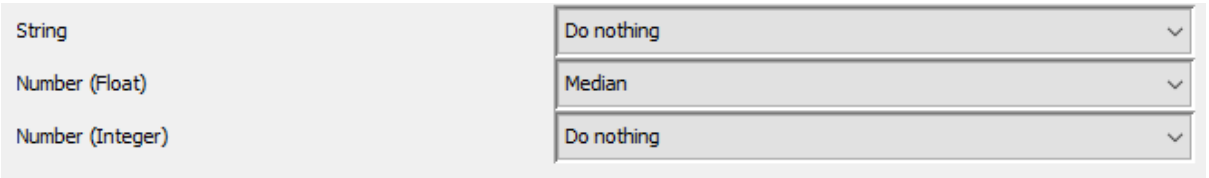
En un primer vistazo observamos que tenemos 100 registro divididos en 9 atributos; diagnóstico, de tipo String, con cuatro valores únicos que puede tomar (HC, Asthma, COPD, Infected); ID, de tipo String con 100 valores , al coincidir con el número de registros nos muestra que cada ID es único por lo tanto no hay valores duplicado; Img_part de tipo float con valores que van desde -330.18 hasta -228; Img_part2 de tipo float con valores desde -321.362 hasta -228; Real_part de tipo float con valores desde -519.19 hasta -442; Real_part2 de tipo float con valores desde -473.929 hasta -442; Género de tipo int con dos valores posibles 0 y 1; Edad de tipo int que va desde 17 años a 86; Fumador de tipo int que tiene tres posibles valores 1,2 y 3.
Tenemos valores faltantes en Img_part , Img_part2, Real_part y Real_part2 que representan un 77% del total de datos.
Observando las estadísticas vemos que la asimetría nos muestra una distribución no simétrica y con la curtosis con valores >3 nos indica colas pesadas, lo que sugiere presencia de outliers por lo que utilizar la media para reemplazar los valores faltantes no es recomendable ya que es sensible a outliers.



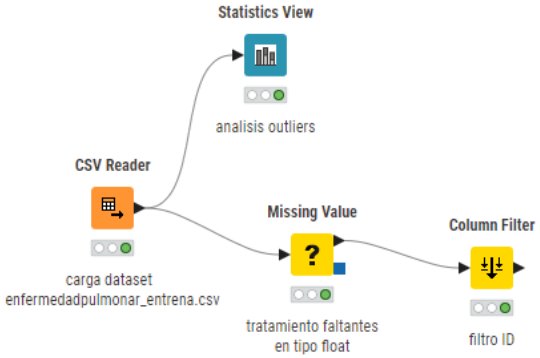
Name	Type	# Missing values	Skewness	Kurtosis
Img_part	Number (Float)	77	2.617	8.016
Img_part2	Number (Float)	77	2.443	7.162
Real_part	Number (Float)	77	-0.89	7.485
Real_part2	Number (Float)	77	1.724	3.708

Dado que los algoritmos de árboles de decisión y random forest (que son el objetivo de este dataset) son robustos ante la presencia de valores faltantes, vamos a tratarlos con la mediana solo con el propósito de no dejar celdas vacías en el dataset. Utilizamos el nodo Missing Values y cambiamos los valores de tipo float por la mediana. Este tratamiento nos permitirá luego evaluar si estos atributos (a pesar del alto porcentaje de imputación) tienen peso predictivo en el modelo final.

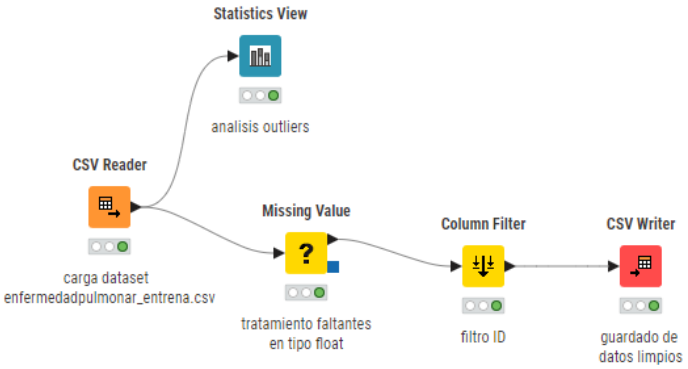




Ahora vamos a filtrar la columna de ID ya que es solo un identificador y no tiene poder predictivo por lo que no es necesario para nuestro objetivo final.

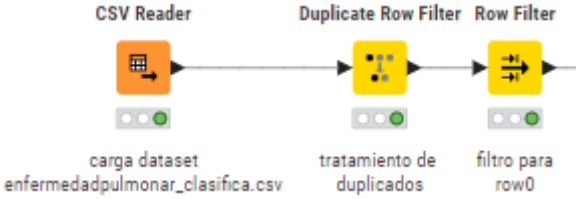


Los algoritmos de árboles y random forest son robustos a los outliers por lo que no los trataremos, además tampoco es necesaria la normalización para estos ya que son modelos basados en reglas de decisión y no es necesario.
Como buena práctica guardamos el nuevo dataset limpio como EPEntrena.csv usando el nodo CSV Writer.

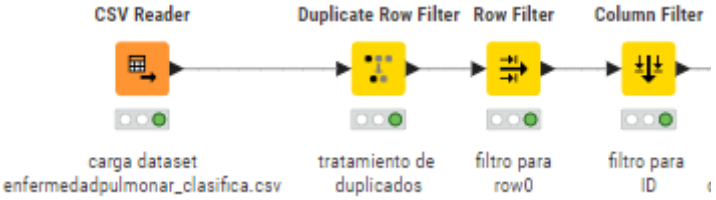


Ahora cargamos el dataset enfermedadpulmonar_clasifica.csv para realizar la limpieza con el nodo CSV Reader.

Tenemos un dataset con 301 registros divididos en 8 atributos; ID de tipo String con 299 valores únicos por lo que ya nos indica columnas duplicadas al no coincidir con la cantidad total de registros; Género de tipo int con dos valores posibles 0 y 1 tambien ; Edad de tipo int de entre 17 y 93 años; Fumador de tipo int con tres valores posible 1, 2 y 3; Imaginary Part de tipo String; Column3 de tipo String; Real Part de tipo String y column5 de tipo String también.
Lo primero que vamos a hacer es filtrar las filas duplicadas guiándonos por el ID; también vamos a filtrar el registro 0 ya que no tiene ningún dato.

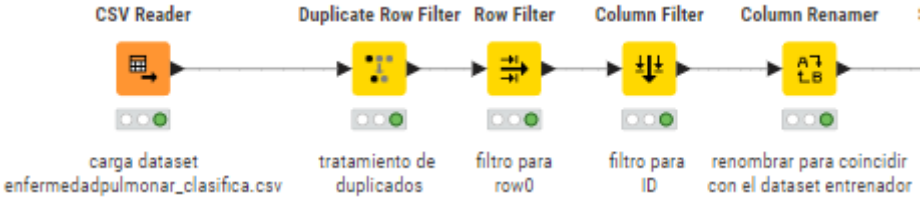


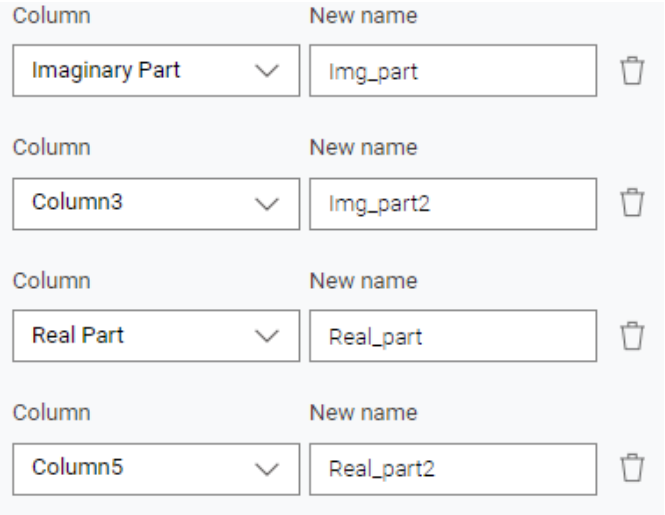
Y así terminamos con 299 registros únicos. Luego filtramos la columna ID ya que no es relevante para nuestro objetivo.



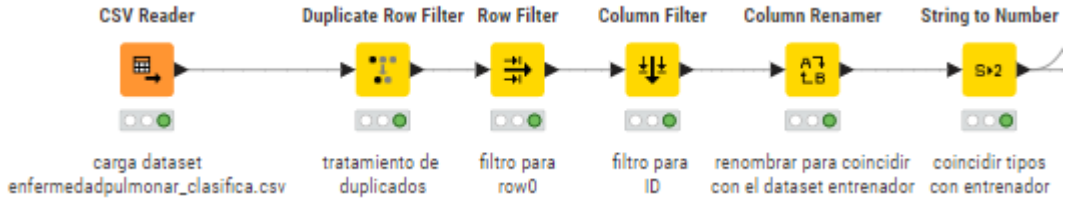
En las columnas de Imaginary Part, Column3, Real Part y Column5 poseemos 222 valores faltantes que representan aproximadamente un 75% del total por lo que debemos tratarlos.
Antes vamos a cambiar el tipo y nombre de estas columnas para que coincidan con el dataset de entrenamiento. El criterio a usar será que la Column3 hace referencia a la columna 3 del dataset de

entrenamiento y la Column5 hace referencia a la columna 5 del dataset de entrenamiento y la columna de Imaginary Part hace referencia a la columna dos y Real_part hace referencia a la columna cuatro.

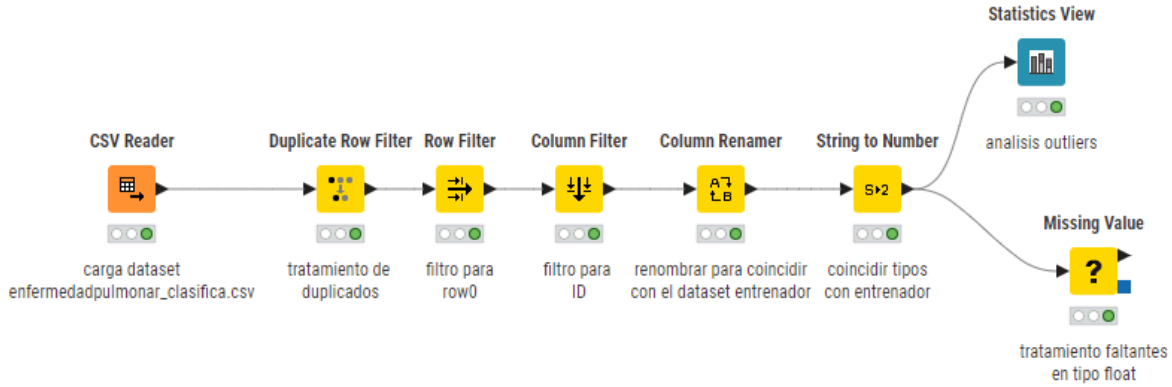




También con el objetivo de coincidir cambiaremos los tipos de String a float.



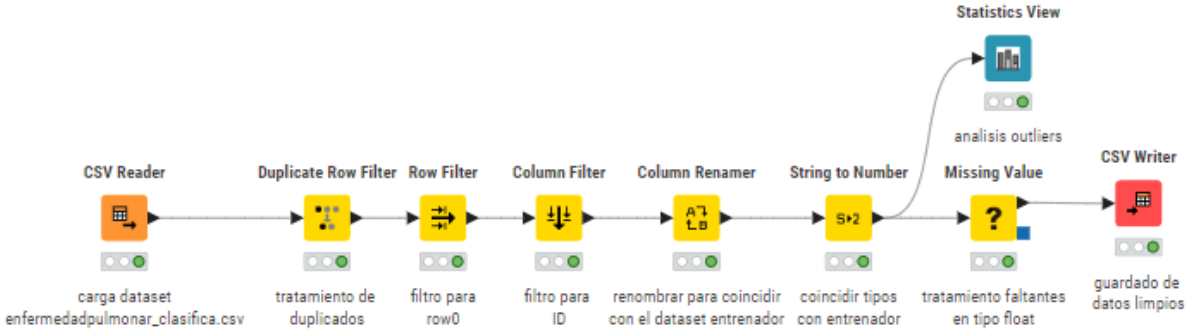
Ahora pasamos a tratar los valores faltantes de la misma forma que en el entrenador ya que las estadísticas nos muestran colas pesadas y una distribución asimétrica. Reemplazamos los valores faltantes por la mediana.



Name	Type	Skewness	Kurtosis
Img_part	Number (Float)	2.348	4.524
Img_part2	Number (Float)	2.07	3.48
Real_part	Number (Float)	5.702	49.035
Real_part2	Number (Float)	7.533	65.57

Number (Float)	Median
Number (Integer)	Do nothing

Guardamos el nuevo dataset limpio como EPClasificar.csv

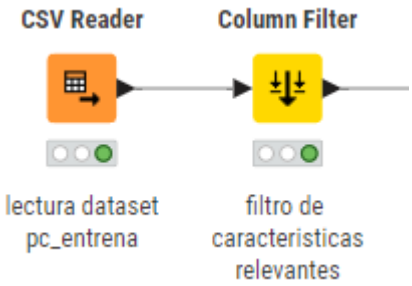


Preprocesamiento dataset de PC

Trabajaremos con el dataset pc_entrena.csv en el cual tenemos datos sobre clasificaciones de tipo laptop o desktop según ciertas características. Nuestro objetivo es clasificar un nuevo conjunto de datos según los datos aportados. Vamos a leer el dataset con un CSV Reader.

En una primera lectura vemos que no tenemos valores faltantes , tenemos 25000 registros divididos en 33 atributos.
Sabemos que estos algoritmos(SVM y K-NN) solo trabajan con valores numéricos por lo que vamos a analizar las columnas String para decidir si transformarlas o filtrarlas en base a nuestro objetivo.
Entre los atributos que tenemos hay varios que no son precisamente para clasificar si una pc es desktop o laptop , las principales características que pueden definir esta clasificación según fuentes externas son el factor de forma, el tamaño de la pantalla, el peso , la fuente de alimentación, el modelo

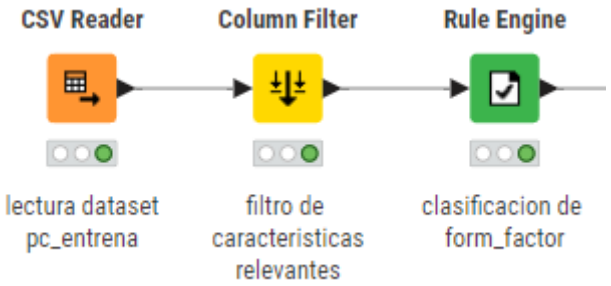
de cpu, la tarjeta gráfica, el tipo de memoria RAM y la conexión de red. Basándonos en eso vamos a tomar las columnas de nuestro dataset que completen esas características.

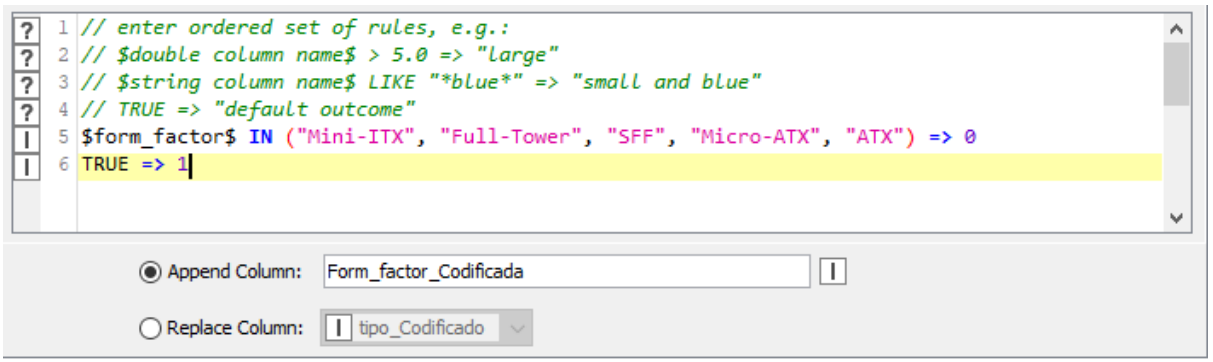


Rows: 14 | Columns: 14

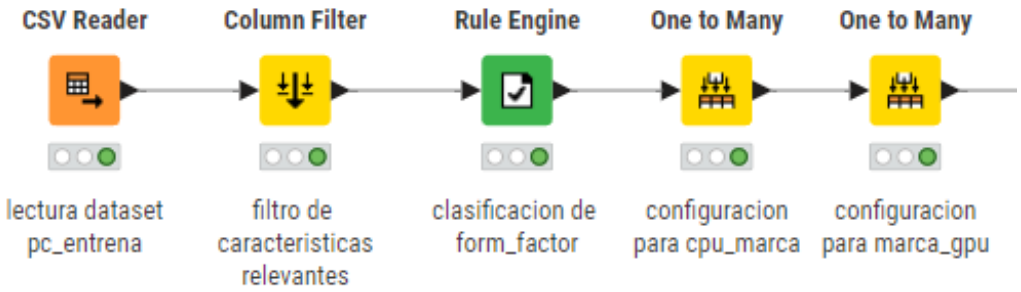
Name	Type	# Missing val...	# Unique val...
tipo	String	0	2
form_factor	String	0	10
cpu_marca	String	0	3
cpu_modelo	String	0	14462
marca_gpu	String	0	4
modelo_gpu	String	0	49
ram_gb	Number (Intege	0	15
pantalla_tamañ	Number (Float)	0	9
battery_wh	Number (Intege	0	8
charger_watts	Number (Intege	0	7
psu_watts	Number (Intege	0	9
wifi	String	0	4
bluetooth	Number (Float)	0	5
peso_kg	Number (Float)	0	47

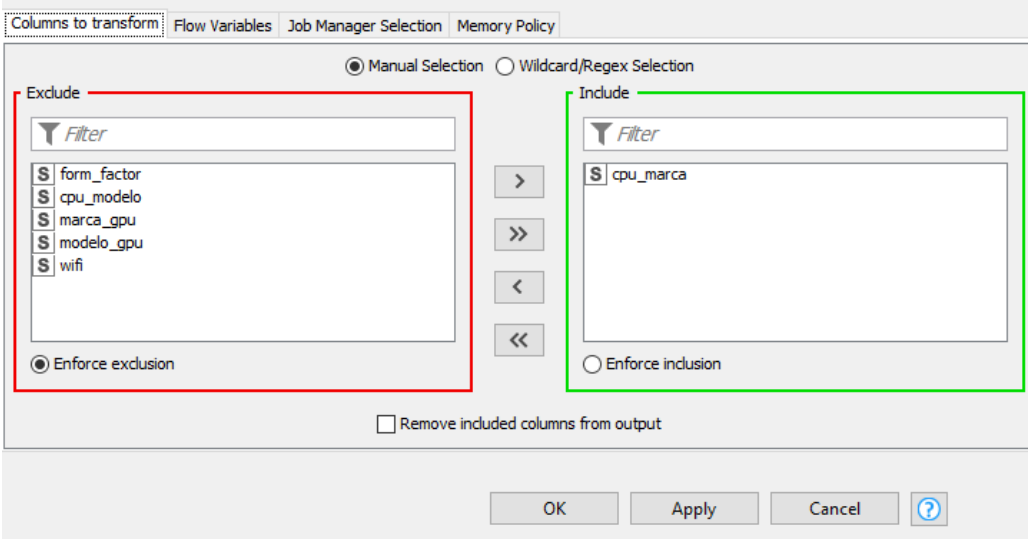
En el atributo form_factor vimos que hay valores que son solo de PC de escritorio (“mini-itx”, “full-tower”, “sff”, “micro-atx”, “atx”) y otros que son solo de laptops (“gaming”, “mainstream”, “ultrabook”, “2-in-1”, “workstation”).
Como es un dato importante, no podemos usar Category to Number, porque ese nodo solo asigna números al azar sin tener en cuenta que unos son desktop y otros laptop.
Entonces lo que hicimos fue crear una nueva columna usando Rule Engine, donde los form_factor de escritorio los pusimos como 0 y los form_factor de laptop los pusimos como 1. Así queda claro qué equipo es desktop y cuál es laptop, y el modelo lo puede usar bien.

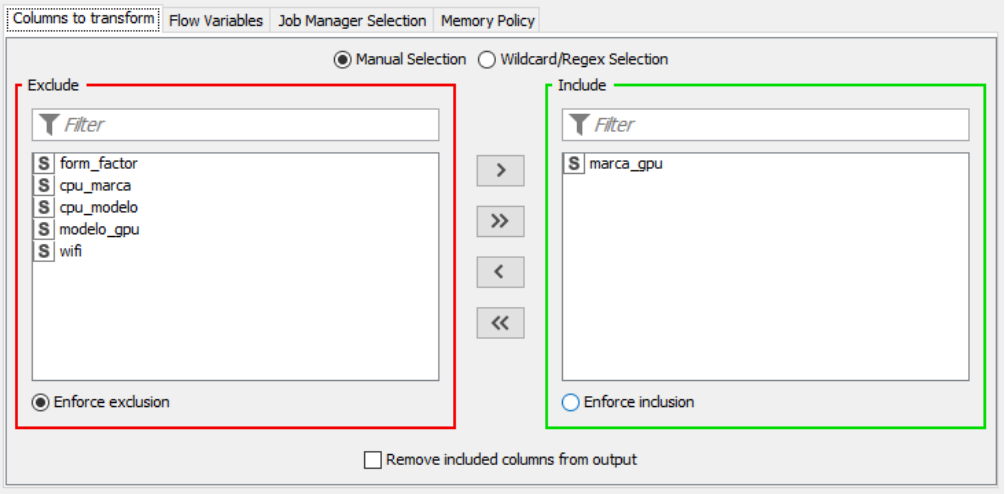




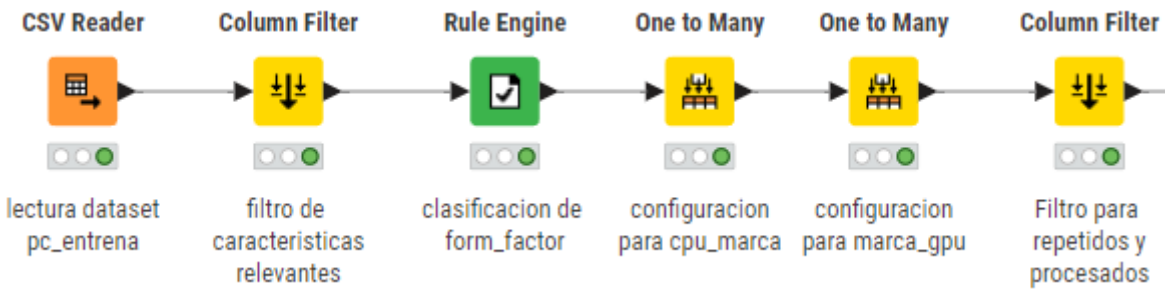
El atributo cpu_marca tiene influencia indirecta sobre la predicción, no se puede saber de qué tipo es una pc con solo la marca pero puede tener relación con otras columnas, como por ejemplo el peso, para establecerlo. Sería incorrecto usar el nodo Category to Number ya que estaría creando un orden que no existe por lo que vamos a usar el nodo One to Many que separara los tres tipos de marca creando columnas separadas y coloca 1 si el equipo pertenece a esa marca o 0 si el equipo no pertenece a esa marca, de esta forma evitamos la categorización invisible. La misma situación pasa con marca_gpu que se divide en cuatro categorías. Por lo que aplicaremos el mismo nodo, aunque esto nos marca un error pero eso se debe a que se repite por ejemplo cpu_marca_AMD y marca_gpu_AMD , por lo que para solucionarlo vamos a usar dos nodos distintos para lo mismo.

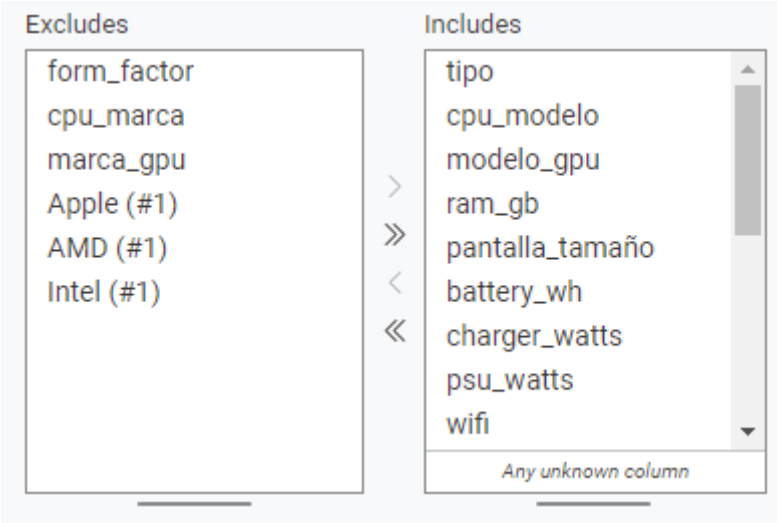




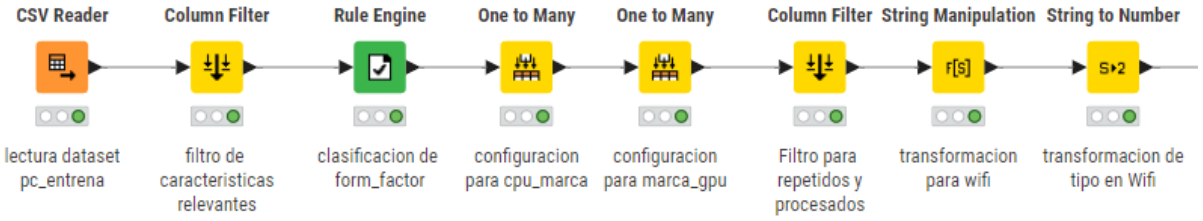


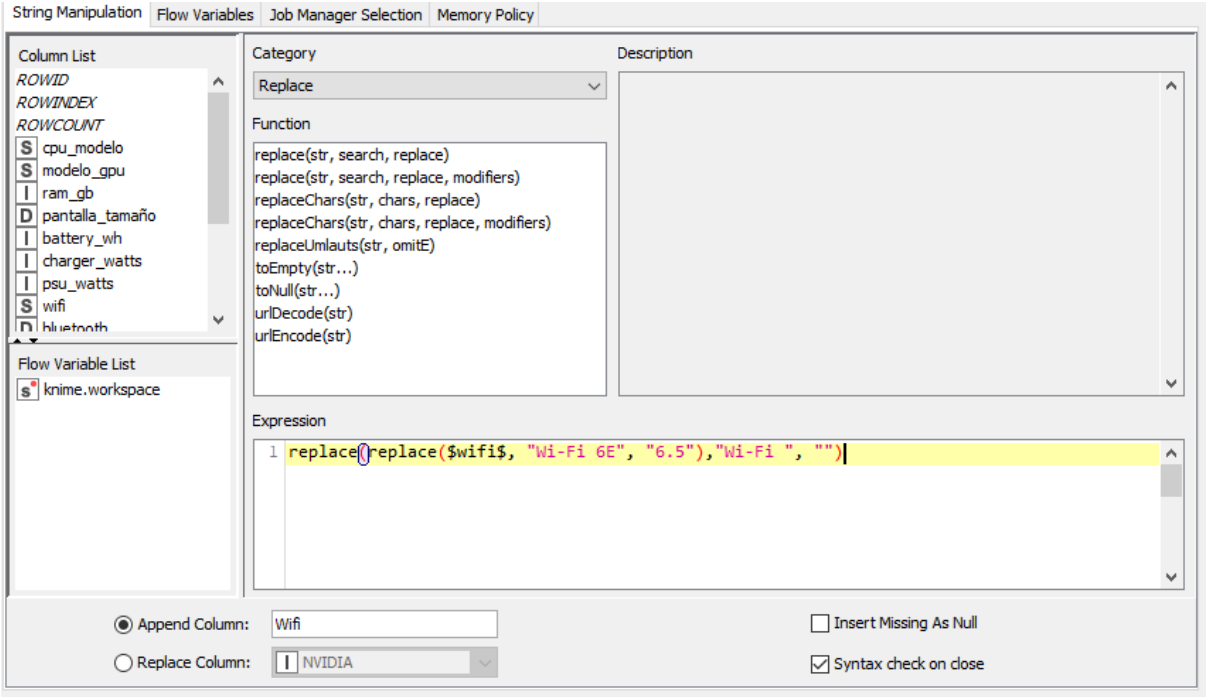
Para reducir la dimensionalidad y hacer que el modelo sea más eficiente vamos a filtrar las columnas de marca_gpu que nos creo y solo quedarnos con la de NVIDIA ya que eso significa que si la columna es 1 el equipo tiene GPU dedicada y si la columna es 0 significa que el equipo tiene GPU integrada. También vamos a filtrar las columnas que ya procesamos como form_factor , cpu_marca y marca_gpu.



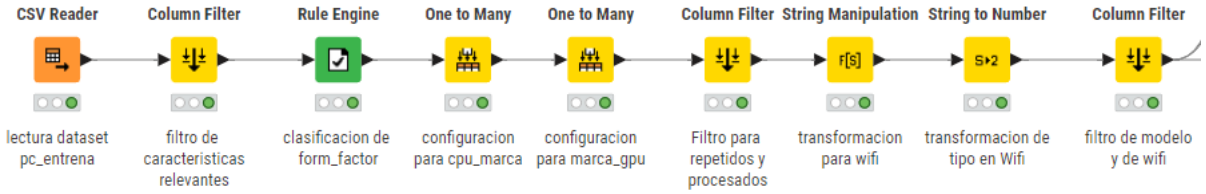


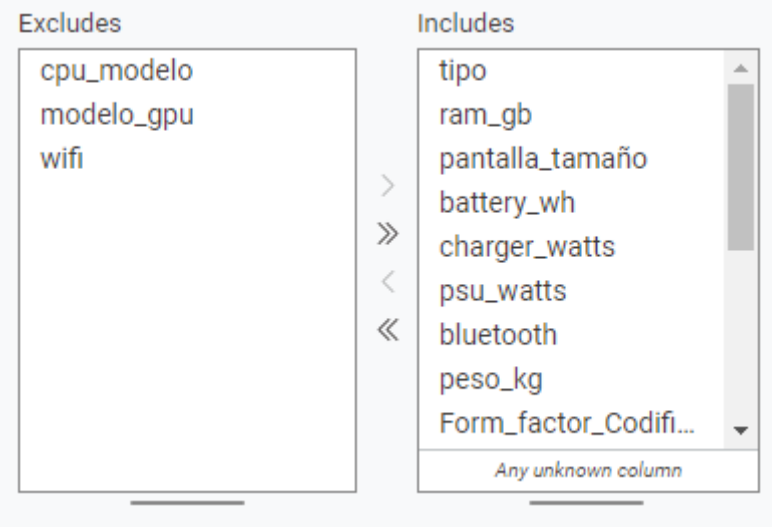
La variable wifi siempre tiene el formato "Wi-Fi + un número" y en algunos casos la letra E. Para cambiar la columna a tipo entero vamos a quitar la palabra “Wi-Fi” y para hacer diferencia entre el 6 y el 6E vamos a reemplazar este último por un .5 y luego transformar la columna a tipo Double.





Las columnas de cpu_modelo y de modelo_cpu tienen muchos datos únicos y nos proporcionan información que ya conseguimos de los demás atributos por lo que vamos a filtrar estas columnas y también la de wifi que ya hemos procesado.





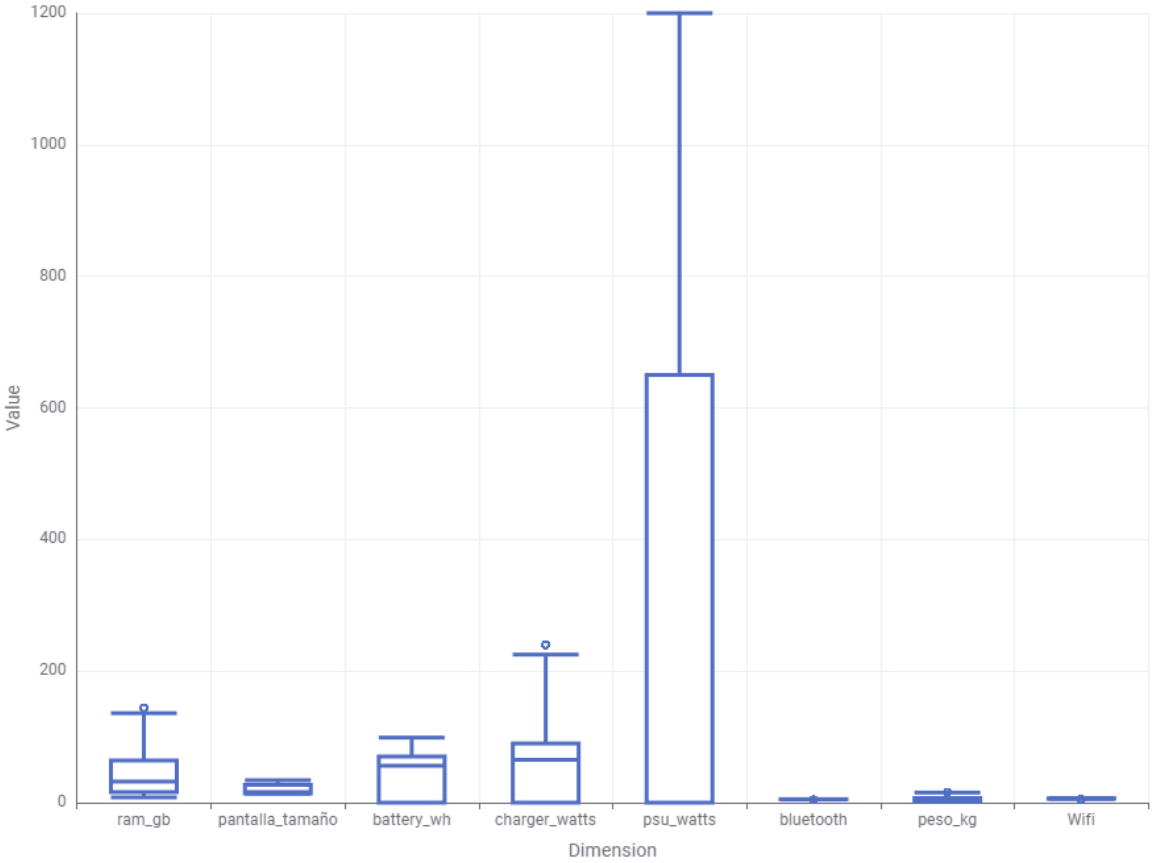
De esta manera nos quedamos con todos los datos numéricos y relevantes para nuestra clasificación sin perder información.

Name	Type	# Missing val...	# Unique val...	Minimum	Maximum
tipo	String	0	2	?	?
ram_gb	Number (Integer)	0	15	8	144
pantalla_tamañ	Number (Float)	0	9	13.3	34
battery_wh	Number (Integer)	0	8	0	99
charger_watts	Number (Integer)	0	7	0	240
psu_watts	Number (Integer)	0	9	0	1,200
bluetooth	Number (Float)	0	5	4.2	5.3
peso_kg	Number (Float)	0	47	0.92	16
Form_factor_Cc	Number (Integer)	0	2	0	1
Desktop	Number (Integer)	0	2	0	1
Laptop	Number (Integer)	0	2	0	1
Intel	Number (Integer)	0	2	0	1
AMD	Number (Integer)	0	2	0	1
Apple	Number (Integer)	0	2	0	1
Desktop (#1)	Number (Integer)	0	2	0	1
Laptop (#1)	Number (Integer)	0	2	0	1
NVIDIA	Number (Integer)	0	2	0	1
Wifi	Number (Float)	0	4	5	7

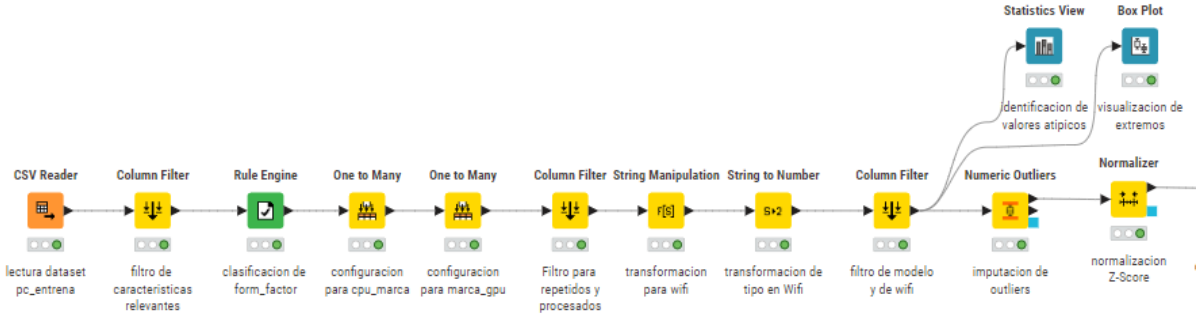
Para los algoritmos que vamos a utilizar es necesario tratar outliers y normalizar; por lo que analizaremos las estadísticas para analizar la presencia de valores atípicos.

Name	Minimum	Maximum	50% Quantile (M...	Standard Deviati...	Variance	Skewness	Kurtosis
ram_gb	8	144	32	31.814	1,012.133	1.056	0.216
pantalla_tamaño	13.3	34	16	6.688	44.731	0.671	-1.072
battery_wh	0	99	56	35.915	1,289.905	-0.135	-1.637
charger_watts	0	240	65	62.696	3,930.75	0.858	0.252
psu_watts	0	1,200	0	355.167	126,143.796	0.775	-0.921
bluetooth	4.2	5.3	5.1	0.25	0.062	-2.585	6.745
peso_kg	0.92	16	2	3.824	14.625	1.199	0.418
Wifi	5	7	6	0.588	0.346	-0.483	-0.49

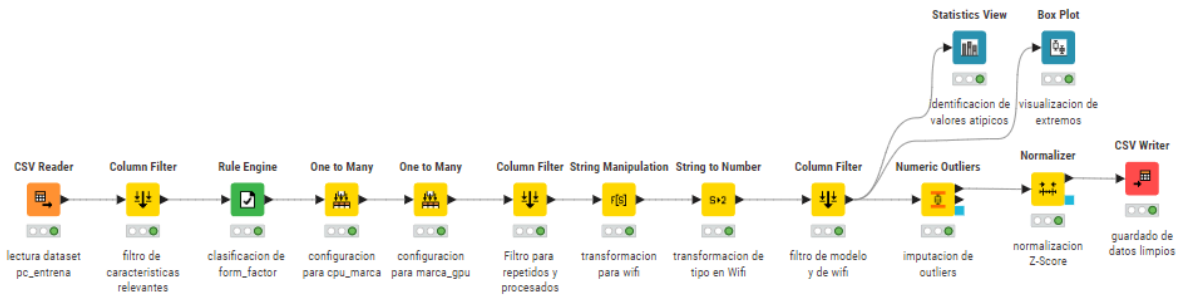
Tenemos valores atípicos en ram_gb ,charger_watts .bluetooth, Wifi y peso_kg,



Con el nodo Numeric Outliers vamos a imputar estos datos reemplazando el extremo por el valor más cercano posible.
Ahora vamos a normalizar. Como nuestro objetivo es predecir con los algoritmos de SVM y de KNN el Z-Score es mejor ya que centra la distribución y en nuestro dataset tenemos muchos valores que son solo 0 y 1 ,para estos va a transformarlos en un valor positivo y uno negativo centrados en 0, a diferencia del min max que mantendrá los valores.

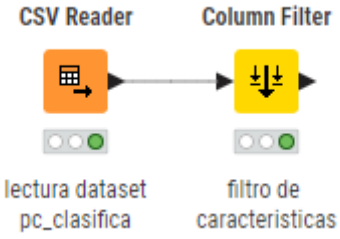


Finalmente guardamos los resultado en un nuevo archivo llamado PCEntrena.CSV



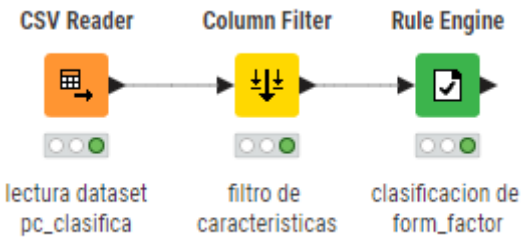
Ahora vamos a realizar el preprocesamiento de datos para el dataset pc_clasificar.csv para que coincida con el dataset pc_entrena que ya procesamos.

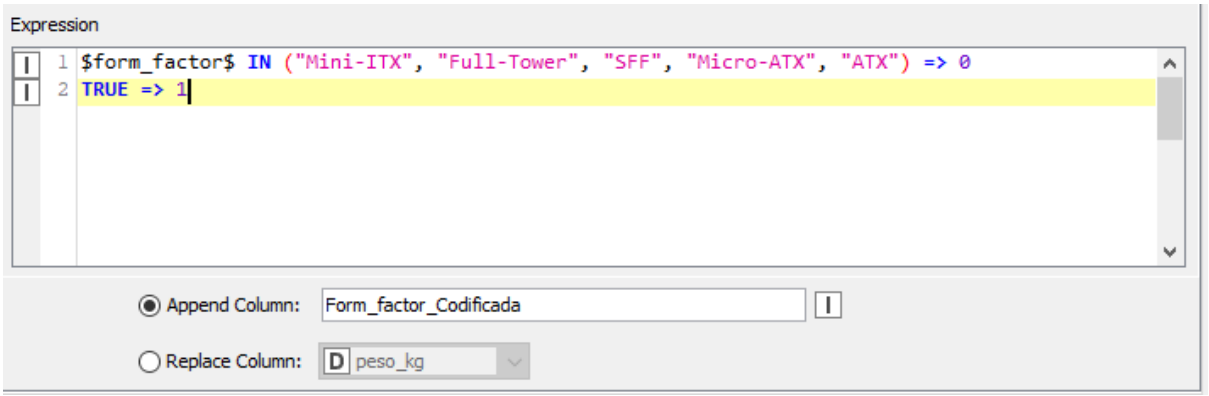
Primero vamos a filtrar las mismas columnas que filtramos para el dataset de entrenamiento.



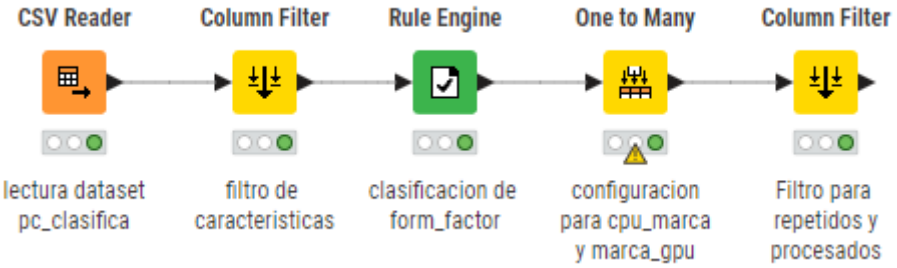
Name	Type	# Missing val...	# Unique val...
form_factor	String	0	10
cpu_marca	String	0	3
cpu_modelo	String	0	14668
marca_gpu	String	0	4
modelo_gpu	String	0	49
ram_gb	Number (Intege	0	15
pantalla_tamañ	Number (Float)	0	9
battery_wh	Number (Intege	0	8
charger_watts	Number (Intege	0	7
psu_watts	Number (Intege	0	9
wifi	String	0	4
bluetooth	Number (Float)	0	5
peso_kg	Number (Float)	0	47

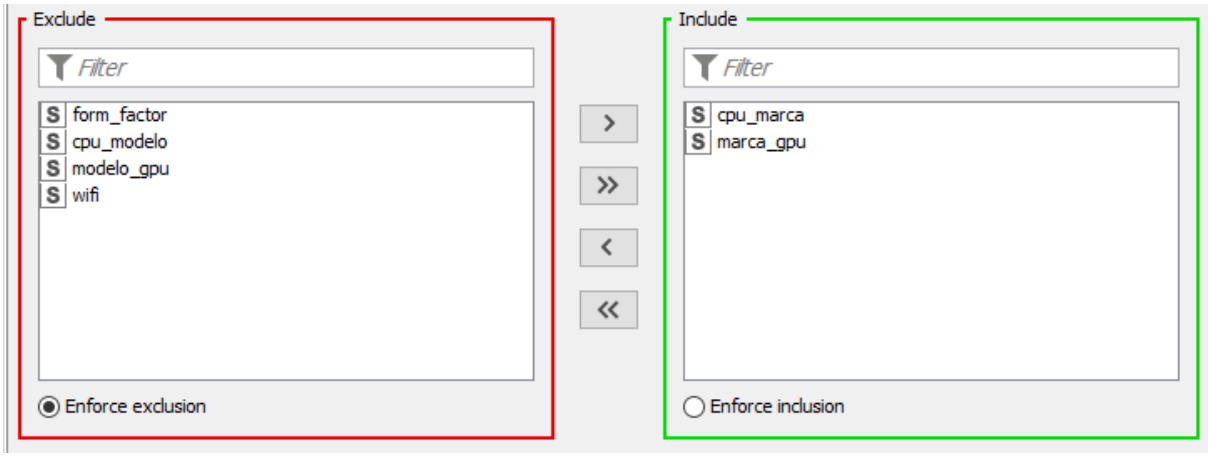
Ahora que tenemos la misma cantidad de columnas vamos a realizar el mismo procesamiento sobre el atributo form_factor para que no haya diferencia.

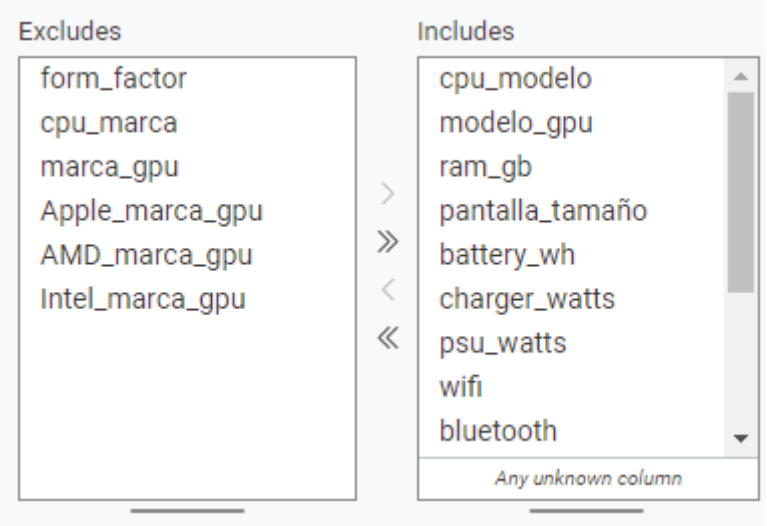




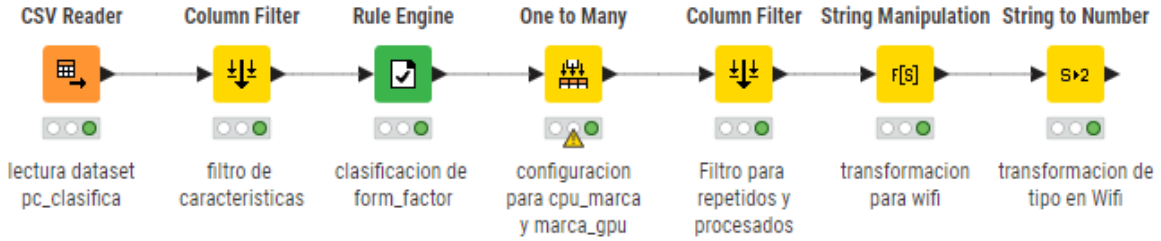
Vamos a convertir las columnas de cpu_marca y marca_gpu a atributos individuales y esta vez vamos a usar un solo nodo porque el error que nos muestra lo solucionaremos con un column filter.



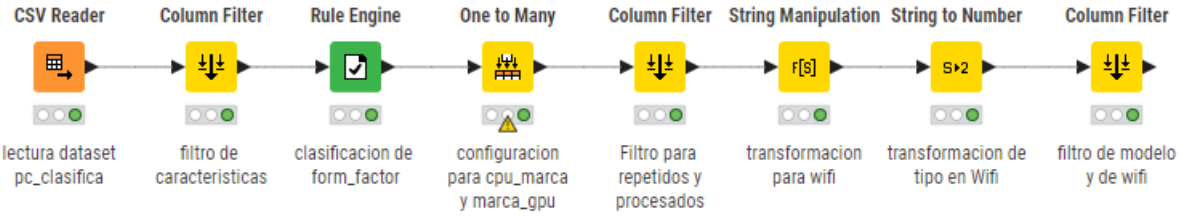




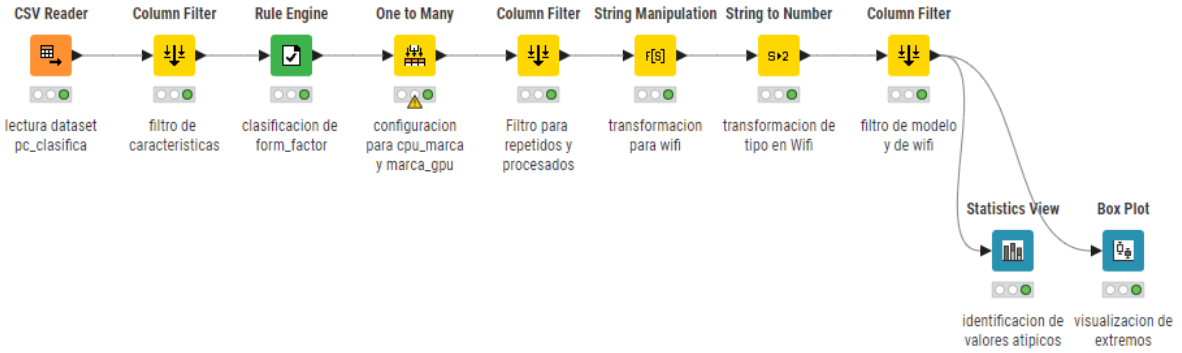
Para el atributo wifi vamos a aplicar los mismo criterios ya que tiene los mismos tipos de datos, primero filtraremos la palabra “Wi-Fi” y luego el 6E lo reemplazamos por un 6.5, y convertiremos la columna a Double.



Ahora filtraremos las columnas que ya procesamos

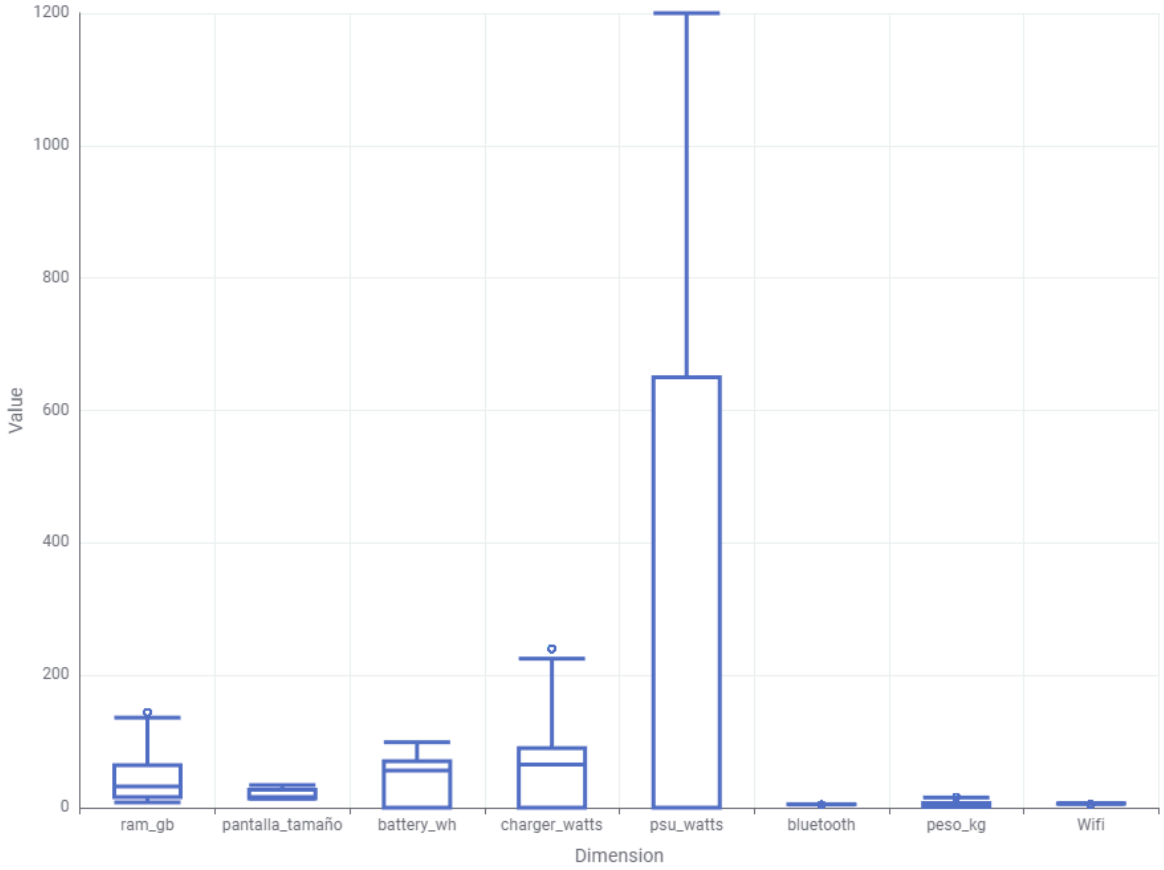


Con los nodos de Statistics View y Box Plot analizaremos la presencia de valores atípicos.

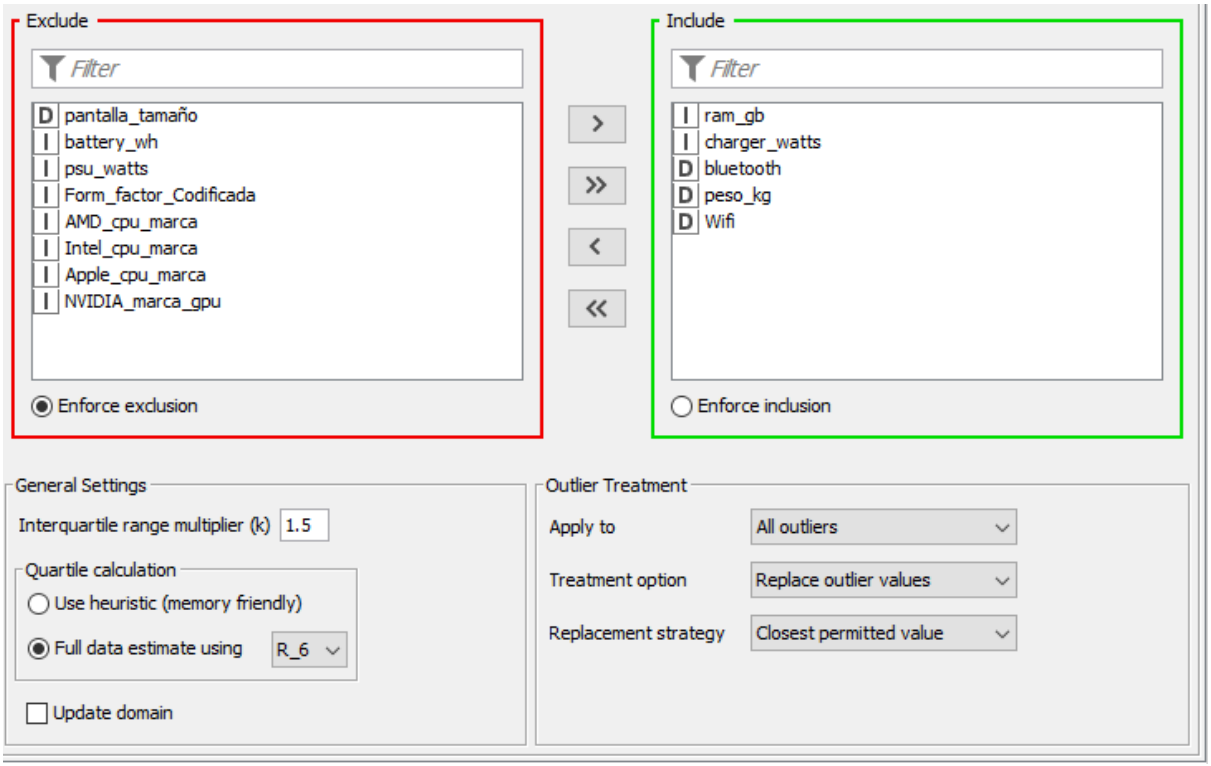


Name	Minimum	Maximum	50% Quantile (M...	Standard Deviati...	Variance	Skewness	Kurtosis
ram_gb	8	144	32	32.104	1,030.694	1.039	0.174
pantalla_tamaño	13.3	34	16	6.724	45.206	0.664	-1.085
battery_wh	0	99	56	35.877	1,287.131	-0.127	-1.635
charger_watts	0	240	65	63.248	4,000.318	0.85	0.205
psu_watts	0	1,200	0	354.362	125,572.611	0.763	-0.949
bluetooth	4.2	5.3	5.1	0.243	0.059	-2.656	7.302
peso_kg	0.92	16	2	3.795	14.401	1.174	0.349
Wifi	5	7	6	0.586	0.344	-0.483	-0.459

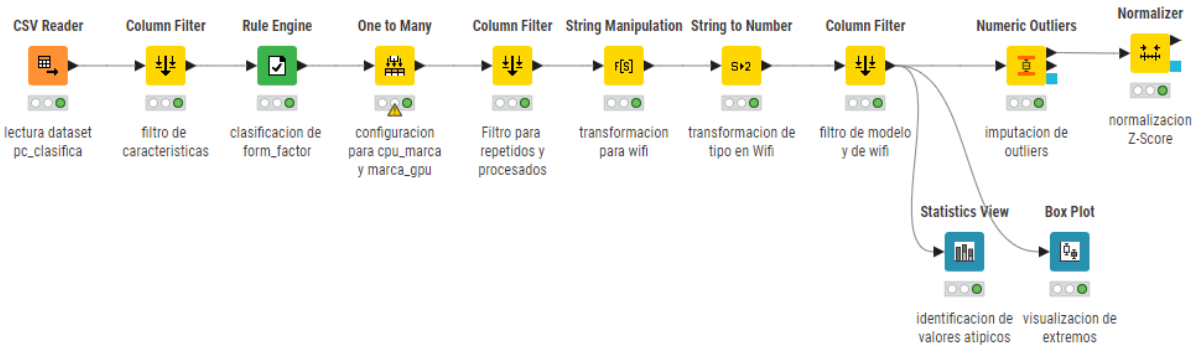
Podemos ver que tenemos valores atípicos en ram_gb , cahrger_watts, bluetooth, peso_kg y Wifi.



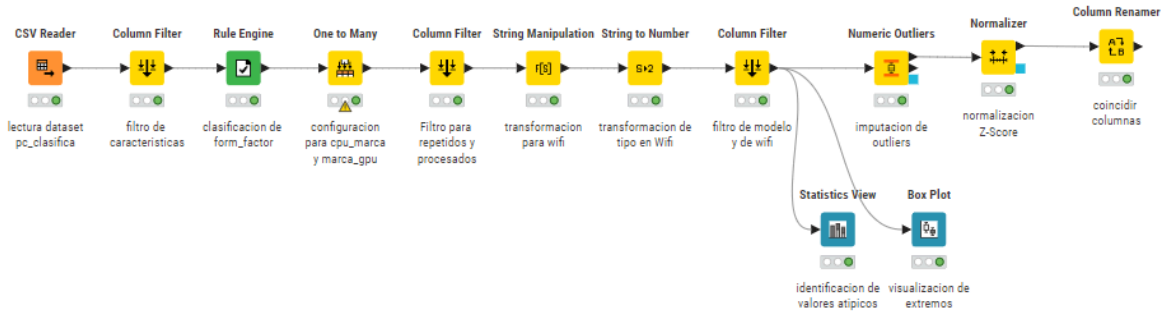
Vamos a tratar estos valores reemplazando por el valor más cercano permitido.

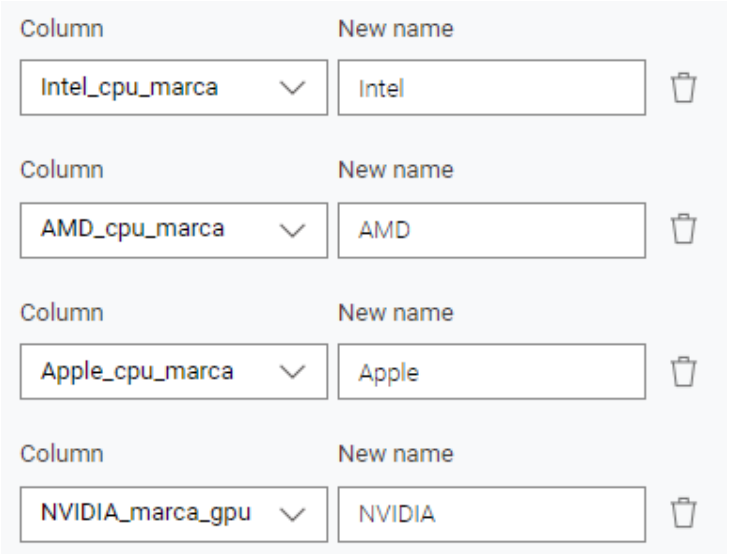


Para normalizar vamos a usar el método de Z-Score con todas las columnas.

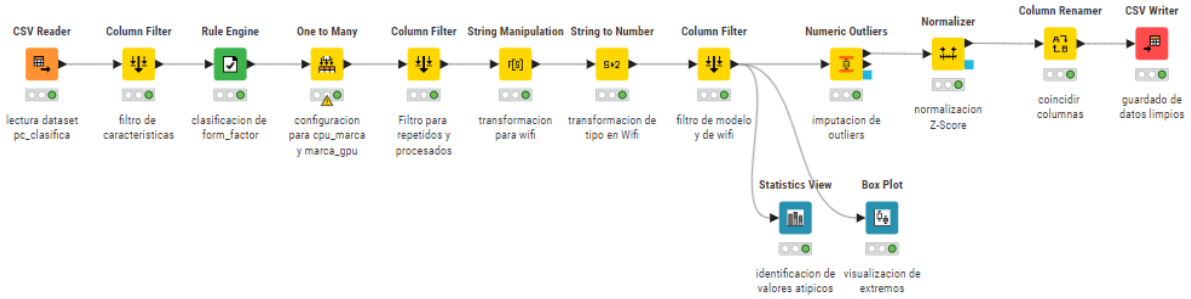


Con un Column Renamer vamos a renombrar las columnas que nos quedaron diferentes al dataset de entrenamiento.





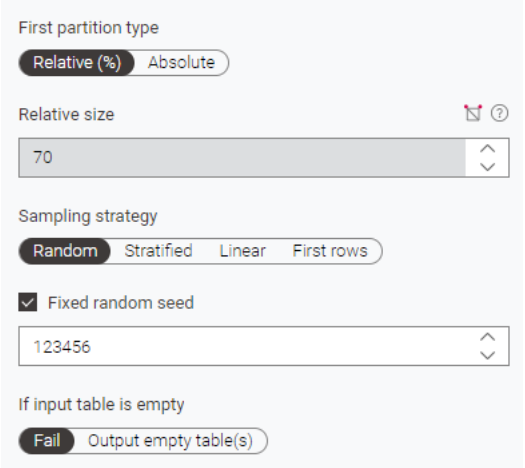
Para finalizar guardamos el nuevo dataset como PCClasificar.csv .

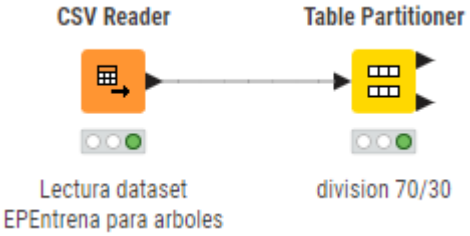


Árbol

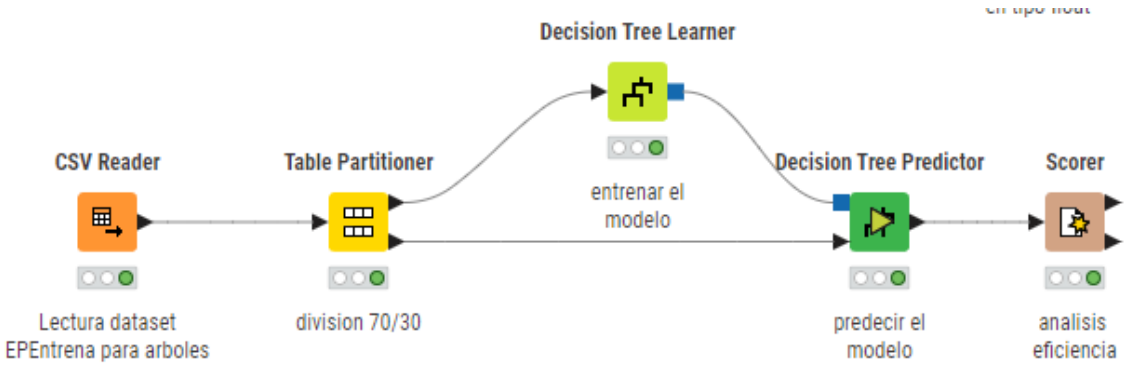
Vamos a cargar el dataset EPEntrena.csv con el nodo CSV Reader. Nuestro objetivo será lograr el modelo más eficiente.

Particionamos el dataset con el Table Partitioner en un 70/30 con una semilla de 123456.





Luego conectamos el nodo Decision Tree Learner con la configuración default para tener una base para empezar a probar y corregir eficiencia.
Conectamos la segunda salida del Table Partitioner al nodo predictor Decision Tree Predictor y luego al nodo Scorer para ver el porcentaje de eficiencia.



El primer resultado obtenido es una eficiencia del 50%, muy bajo.

File Hilite

Diagnostic...	COPD	HC	Infected	Asthma
COPD	4	1	0	0
HC	0	10	1	0
Infected	0	6	1	0
Asthma	3	4	0	0

Correct classified: 15 Wrong classified: 15

Accuracy: 50% Error: 50%

Cohen's kappa (κ): 0,274%

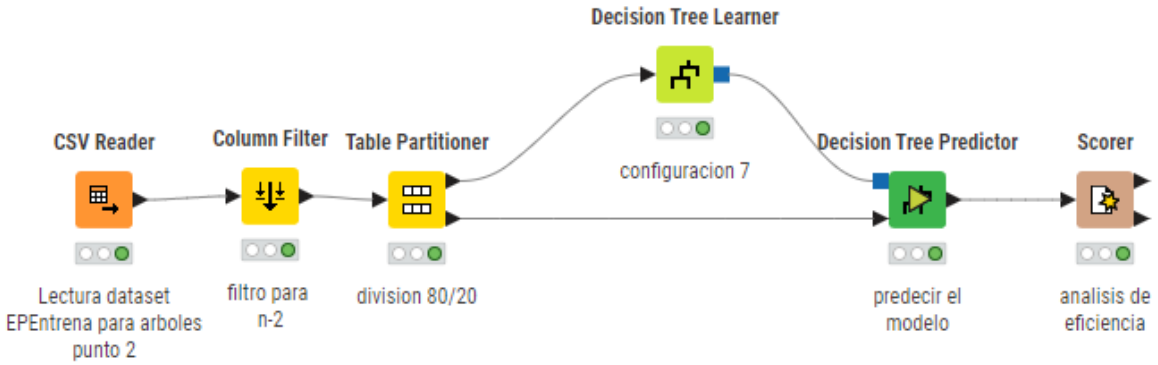
Por lo que vamos a configurar el nodo de Decision Tree Learner para subir este porcentaje.
Primero, como nuestro dataset es de menos de 1000 registros el número mínimo de registros por nodo debe ser de entre 5 y 10 , y nuestra configuración original era de 2 por lo que haremos pruebas dentro de ese rango.

Knime	Partición	Quality index	Pruning method	min num records per node	Accuracy
configuración original	70/30	Gini index	no pruning	2	50%
configuración 1	70/30	Gini index	no pruning	5	43,30%
configuración 2	70/30	Gini index	no pruning	10	50%
configuración 3	70/30	Gini index	no pruning	8	53,30%
configuración 4	70/30	Gini index	MDL	8	50%
configuración 5	70/30	Gain ratio	no pruning	8	53,30%
configuración 6	70/30	Gain ratio	MDL	8	50%
configuración 7	80/20	Gini index	no pruning	8	65%
configuración 8	80/20	Gini index	MDL	8	65%
configuración 9	80/20	Gain ratio	no pruning	8	65%
configuración 10	80/20	Gain ratio	MDL	8	50%
configuración	80/20	Gini index	no pruning	10	65%

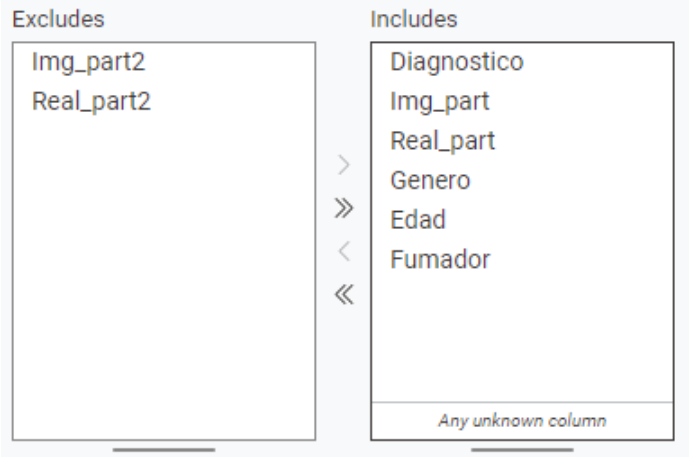
11					
configuración 12	80/20	Gini index	MDL	10	65%
configuración 13	80/20	Gain ratio	no pruning	10	65%

Alcanzamos una eficiencia máxima del 65%, por lo que optamos por la configuración 7. Esta elección se debe a que la partición 80/20 demostró ser más efectiva al tener un conjunto de datos de pocos registros; Gini index frente a Gain ratio ya que ambos alcanzaron el mismo nivel de eficiencia pero el gini index es más simple ya que no usa logaritmos para su clasificación; y la decisión de no podar se basa en que el control de crecimiento del árbol ya fue logrado mediante el número mínimo de registros por nodo de 8 y con eso ya evitamos el sobreajuste. Podar bajo el rendimiento en la mayoría de las configuraciones porque eliminó criterios de división en el árbol.

El objetivo ahora es ver si la eliminación de atributos 65% de exactitud obtenido anteriormente. Esto lo haremos mediante el nodo Columns Filter agregándolo luego del CSV Reader.



Actualmente disponemos de siete atributos, excluyendo el atributo diagnóstico, el cual es el que deseamos predecir. Como en nuestra limpieza los datos ruidosos venían de img y real vamos a filtrar estos datos para ver si sube la eficiencia.



Con esta nueva base obtenemos una eficiencia del 65%.

File Hilite

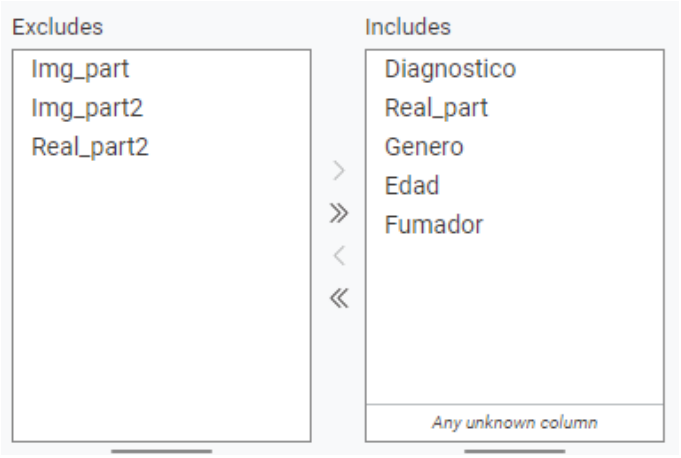
Diagnostic...	COPD	HC	Asthma	Infected
COPD	4	1	0	0
HC	0	9	0	0
Asthma	0	2	0	0
Infected	0	2	2	0

Correct classified: 13 Wrong classified: 7

Accuracy: 65% Error: 35%

Cohen's kappa (κ): 0,44%

Vamos a sacar el atributo Img_part y quedarnos con n-3 atributos.



Obtenemos una eficiencia del 65%.

File Hilite

Diagnostic...	COPD	HC	Asthma	Infected
COPD	4	1	0	0
HC	0	9	0	0
Asthma	0	2	0	0
Infected	0	2	2	0

Correct classified: 13 Wrong classified: 7

Accuracy: 65% Error: 35%

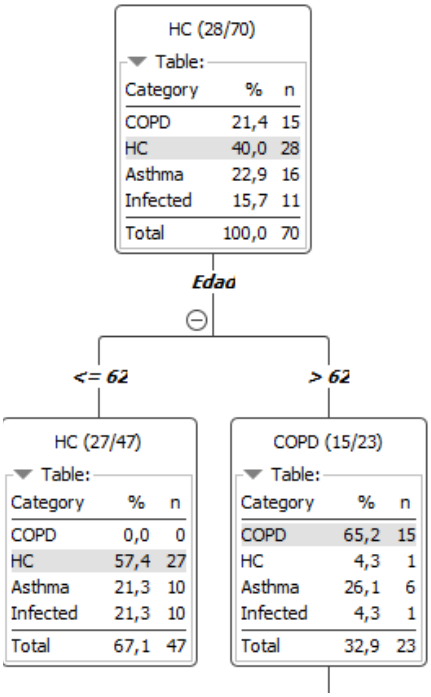
Cohen's kappa (κ): 0,44%

La eliminación de atributos no mejoró la eficiencia. Esto nos dice que ya habíamos llegado a la mejor configuración y que el ruido no afectaba nuestros resultados.

El nivel de eficiencia más alto obtenido con el algoritmo de Árbol de Decision fue del 65% de exactitud. Esto se traduce en que cada 100 pacientes de un conjunto, el algoritmo predice

correctamente el diagnóstico de enfermedad pulmonar en 65 ocasiones, este valor es moderado o bajo y es el límite de eficiencia que podemos obtener con este conjunto de datos.

El atributo que tiene mayor peso en el momento de la clasificación es la edad.

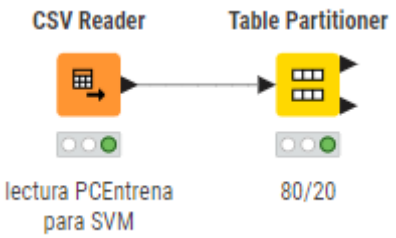


Este valor no puede modificarse manualmente, ya que los árboles de decisión determinan automáticamente qué atributo utilizar en cada división del árbol. El algoritmo selecciona en cada nodo la variable que genera la mayor reducción del desorden o el mayor aumento en la pureza. Por ello, la edad fue elegida como el atributo más relevante porque es el que mejor separa las clases dentro del conjunto de datos.

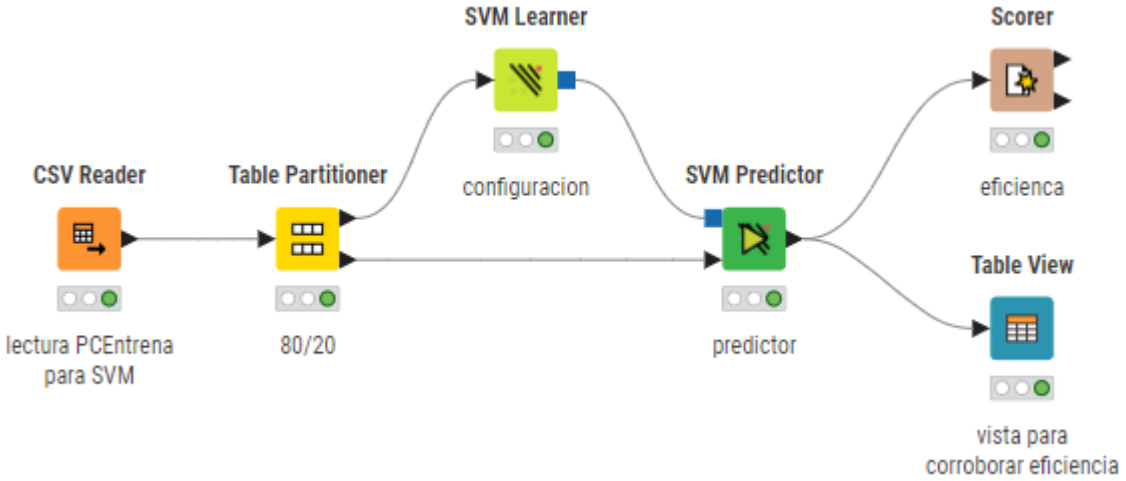
SVM

Vamos a trabajar con el dataset PCEntrena que ya hemos preprocesado.

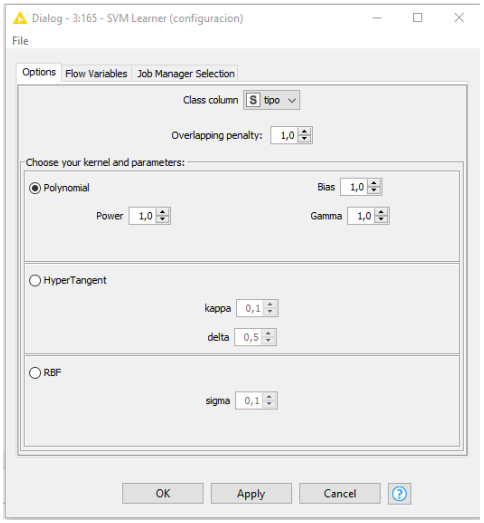
Aplicamos el nodo Partitioner con una división 80/20 y una semilla de 123456.

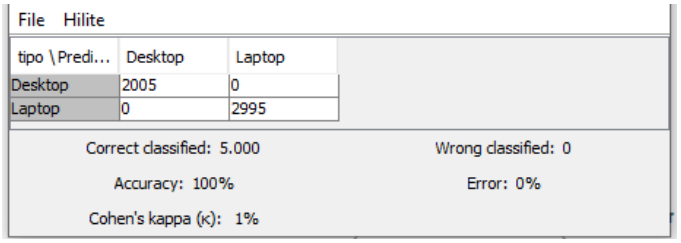


Y usamos el nodo SVM Learner y Predictor con las configuraciones default para ver con el nodo Scorer el porcentaje de eficiencia.

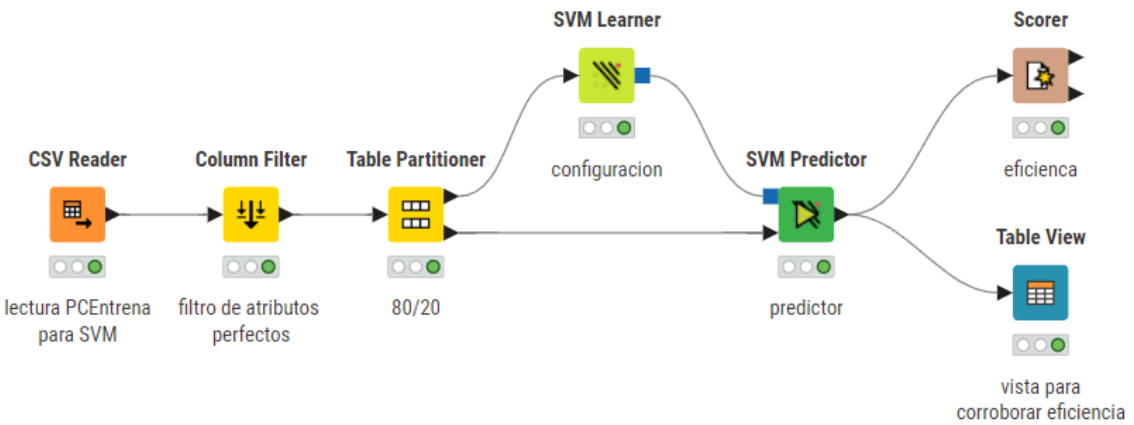


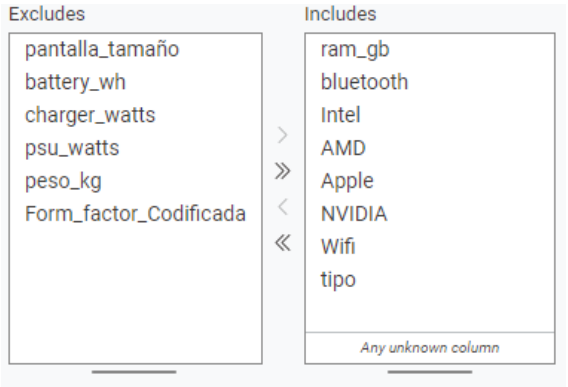
La configuración default con la que viene el nodo es con el kernel Polynomial, esta configuración nos da un 100% de eficiencia.

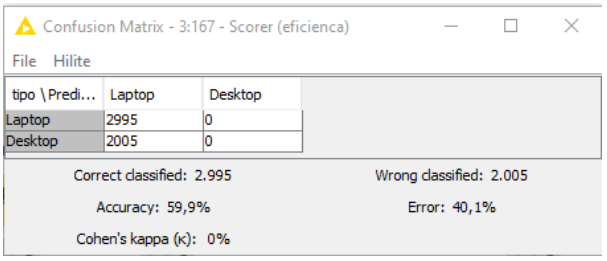




Este 100% de eficiencia con el kernel Polynomial puede deberse a que los atributos están altamente preprocesados por lo que seguramente haya una combinación que es capaz de separar las clases de tipo de forma perfecta. Viendo nuestro dataset sabemos que hay atributos que distinguen fácilmente el tipo como psu_watts, peso_kg, form_factor, pantalla_tamaño , battery_wh y charger_watts. Si filtramos estas columnas podemos ver como con el mismo kernel la eficiencia baja a un 59% y además no está clasificando correctamente Desktop.







Estos atributos tienen un peso altísimo y son los principales discriminadores.
Las características que definen a un Desktop son extremas: altos psu_watts y peso_kg y bajos battery_wh y charger_watts. Al filtrar estas columnas, el modelo se ve obligado a usar características menos definitorias por lo que la clase “Desktop” pierde su patrón único y el modelo comienza a clasificar incorrectamente.
Ahora vamos a volver a sumar todos los atributos y probar con los distintos tipos de kernels para ver si se mantienen esto.

KERNEL	Eficiencia
Polynomial	100%
HyperTangent	100%
RBF	67,18%

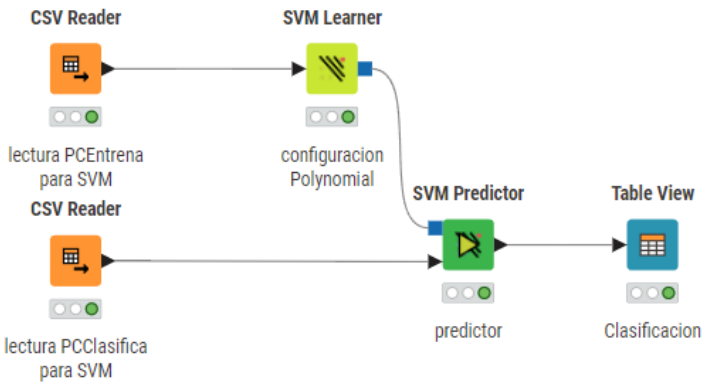
La diferencia de rendimiento entre los kernels nos confirma que la línea de separación de los datos es simple y que la mejor solución es el modelo menos complejo que logra el 100% de eficiencia.
El kernel RBF es bastante complejo. Si lo dejamos con la configuración por defecto, el modelo puede fallar. Por otro lado, los kernels Polynomial e HyperTangent son más sencillos y llegan a la solución más rápido. Esto nos muestra que, para este problema, no hace falta mapear a un espacio de dimensiones infinitas como lo hace el RBF. Por lo tanto, los modelos más eficientes para este caso son los de Polynomial e HyperTangent.

Ahora vamos a realizar el mismo procedimiento pero sin normalizar los datos para comparar resultados. Vamos a guardar una copia del dataset en nuestro preprocesamiento antes de normalizar y lo llamaremos PCEntrenaSinNorm.

KERNEL	Eficiencia sin Normalizar
Polynomial	100%
HyperTangent	error
RBF	73,46%

El kernel HyperTangent resultó ser menos práctico porque necesita la normalización para funcionar bien, algo que no le pasa al kernel Polynomial. Este detalle muestra que el kernel HyperTangent es muy sensible y requiere un preprocesamiento cuidadoso. Por ende podemos decir que el kernel más eficiente para este dataset es el Polynomial ya que no requeriría un preprocesamiento estricto, por ende es más sencillo, y nos ofrece una eficiencia del 100%.

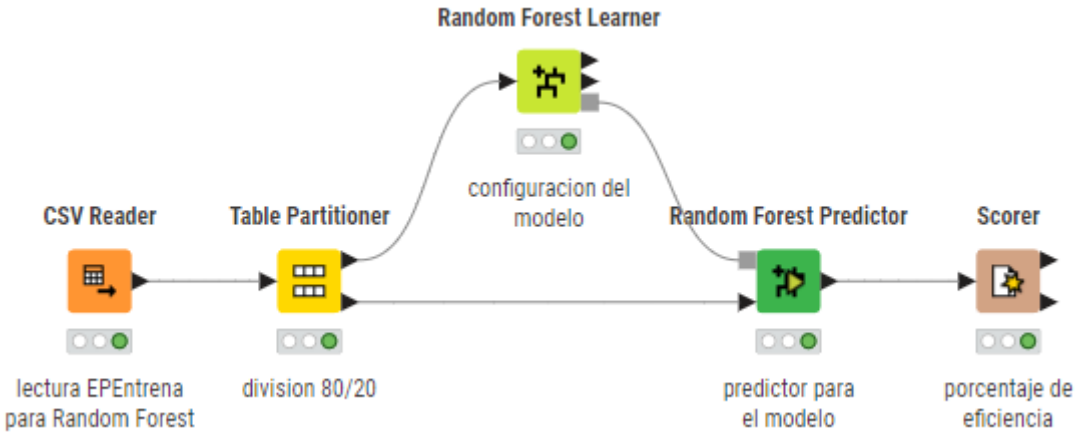
Por último vamos a realizar la clasificación basada en el modelo definido con el dataset PCClasifica.



ram_gb Number (Float)	pantalla_t... Number (Float)	battery_wh Number (Float)	charger_w... Number (Float)	psu_watts Number (Float)	bluetooth Number (Float)	peso_kg Number (Float)	Form_fact... Number (Float)	AMD Number (Float)	Intel Number (Float)	Apple Number (Float)	NVIDIA Number (Float)	Wifi Number (Float)	Prediction... String
1.747	1.022	-1.166	-0.99	1.348	-0.784	2.313	-1.221	1.35	-1.06	-0.364	-1.11	1.863	Desktop
0.999	-0.912	0.507	0.059	-0.769	-0.784	-0.717	0.819	-0.741	0.943	-0.364	0.901	1.863	Laptop
-0.248	-0.912	0.785	0.462	-0.769	-0.103	-0.823	0.819	1.35	-1.06	-0.364	-1.11	0.87	Laptop
-0.248	1.765	-1.166	-0.99	0.078	-0.784	-0.072	-1.221	-0.741	0.943	-0.364	-1.11	-0.124	Desktop
0.75	0.576	-1.166	-0.99	1.066	0.578	0.988	-1.221	1.35	-1.06	-0.364	0.901	-0.124	Desktop
-0.248	-0.674	1.343	1.915	-0.769	-0.103	-0.735	0.819	-0.741	0.943	-0.364	0.901	-0.124	Laptop
0.75	-0.674	0.507	1.915	-0.769	0.578	-0.735	0.819	-0.741	0.943	-0.364	0.901	-0.124	Laptop
-0.746	1.022	-1.166	-0.99	1.348	-0.103	0.723	-1.221	1.35	-1.06	-0.364	0.901	-0.124	Desktop
-0.248	-0.912	1.343	1.915	-0.769	0.578	-0.823	0.819	-0.741	0.943	-0.364	0.901	-0.124	Laptop
2.744	0.576	-1.166	-0.99	0.078	1.259	0.988	-1.221	1.35	-1.06	-0.364	0.901	0.87	Desktop
1.747	2.063	-1.166	-0.99	1.066	0.578	-0.072	-1.221	-0.741	-1.06	2.751	-1.11	0.87	Desktop
0.75	-0.912	0.507	1.915	-0.769	1.259	-0.505	0.819	-0.741	0.943	-0.364	0.901	-0.124	Laptop

Random Forest

Nuestra variable objetivo (target) será Diagnostico. Nuestro flujo de datos será el siguiente en el cual iremos haciendo las modificaciones de configuraciones indicadas:



Para el nodo Partitioner usaremos la configuración que nos dio mayor eficiencia para árboles de decisión que fue la división de 80/20 ya que es un dataset de pocos registros. Realizaremos una comparación entre las diferentes configuraciones que podemos realizar sobre el nodo Random Forest Learner y nos quedaremos con la configuración que nos de mayor eficiencia.. Para la cantidad mínima de elementos por nodo vamos a poner el número acordado de cinco. Y vamos a incluir todas las columnas. Tampoco vamos a modificar la profundidad del árbol, dejaremos el default.

Criterio de separación	Cantidad de modelos	Exactitud
Information Gain Ratio	10	55%
	25	60%
	50	60%
	100	55%
Information Gain	10	65%
	25	65%
	50	65%
	100	55%
Gini Index	10	65%
	25	65%

	50	65%
	100	55%

La máxima eficiencia de clasificación alcanzada fue del 65%. Esta precisión se logró con configuraciones basadas tanto en el criterio Information Gain como en el Gini Index, ambas utilizando 10, 25 o 50 modelos de conjunto.
Para seleccionar la mejor configuración, se aplicó un criterio de optimización de recursos, eligiendo la que logra la máxima precisión con el menor número de modelos. Por lo tanto, la configuración seleccionada como la más eficiente es la basada en el Gini Index con 10 modelos, ya que ofrece el 65% de precisión minimizando el costo computacional.

El problema que igualmente podemos observar es que aunque tenga una eficiencia del 65% nunca clasifica nada hacia Asthma o hacia Infected ,todo lo clasifica en HC o COPD, lo mismo que pasaba en los árboles de decisión. Si observamos la tabla podemos ver que ambas enfermedades las clasifica como HC.

File Hilite

Diagnostic...	COPD	HC	Asthma	Infected
COPD	4	1	0	0
HC	0	9	0	0
Asthma	0	2	0	0
Infected	0	4	0	0

Correct classified: 13 Wrong classified: 7

Accuracy: 65% Error: 35%

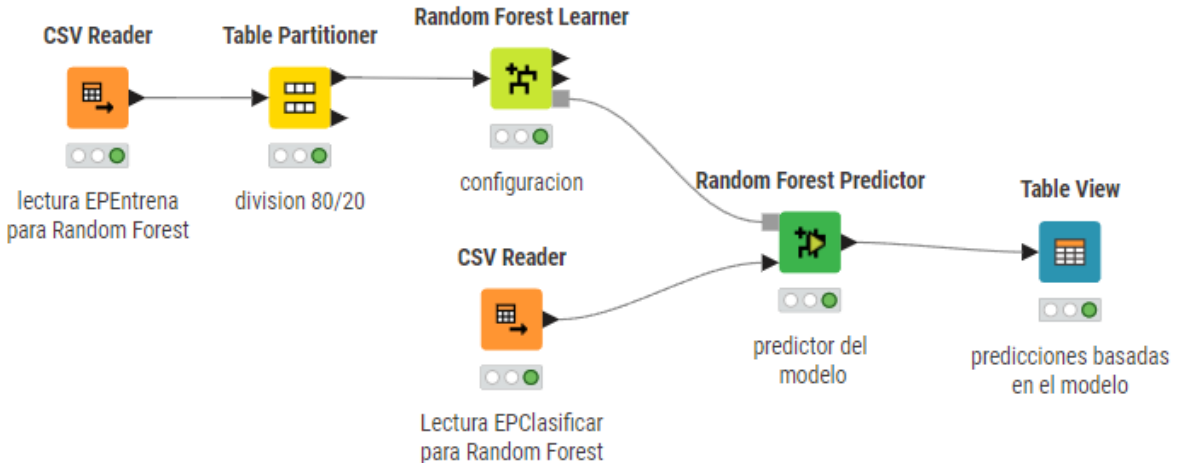
Cohen's kappa (κ): 0,407%

Esto puede deberse a que en nuestro dataset de entrenamiento hay pocos registros para clasificar estas dos enfermedades , nuestra variable objetivo tiene una distribución desigual de las clases. El modelo ignora estas clases porque es menor el error de clasificarlos mal en HC que el error de clasificarlo mal en su propia clase. Esto nos hace pensar que quizás no es recomendable guiarnos por el porcentaje de acierto sino por las clasificaciones como en este caso que clasifica correctamente 13 sobre 20. Por eso mismo también cuando le pasamos el dataset de clasificar y lo clasifica hay valores de confianza muy bajo.

RowID	Img_part Number (Float)	Img_part2 Number (Float)	Real_part Number (Float)	Real_part2 Number (Float)	Genero Number (Integ...	Edad Number (Integ...	Fumador Number (Integ...	Prediction... String	Predic... Number (Float)
Row20	-231	-231	-441	-441	1	73	1	COPD	0.4
Row55	-323	-314.487	-473.73	-466.981	0	60	2	HC	0.4
Row23	-323	-314.487	-473.73	-466.981	0	61	2	HC	0.4
Row0	-320.61	-300.564	-495.26	-464.172	1	77	2	COPD	0.5
Row7	-315.82	-294.27	-471.34	-463.193	0	56	2	HC	0.5
Row11	-323	-293.029	-466.55	-431.991	1	93	2	COPD	0.5
Row22	-231	-231	-442	-442	1	78	2	COPD	0.5
Row24	-320.61	-311.177	-473.73	-468.304	1	64	2	COPD	0.5
Row29	-332.57	-319.037	-473.73	-463.212	0	54	2	HC	0.5
Row43	-323	-314.487	-473.73	-466.981	1	63	3	COPD	0.5
Row45	-323	-314.487	-473.73	-466.981	1	81	1	COPD	0.5
Row62	-323	-314.487	-473.73	-466.981	0	62	1	HC	0.5
Row68	-323	-314.487	-473.73	-466.981	0	61	1	HC	0.5
Row71	-303.86	-288.044	-514.4	-453.225	0	50	2	HC	0.5
Row77	-327.78	-318.567	-507.23	-469.587	1	44	1	HC	0.5

El modelo Random Forest se basa en un mecanismo de votación. Si se asume que hubo 4 votos para HC, 3 para COPD y 3 para Asthma, el diagnóstico resultante es HC. Sin embargo, esto no implica una mayoría absoluta. Aunque el valor de confianza sea de 0.4, el 60% restante del bosque no estuvo de acuerdo con esta clasificación. A pesar de esto, se asigna el diagnóstico de HC debido a que obtuvo la mayor cantidad de votos, lo que puede conducir a un diagnóstico erróneo.

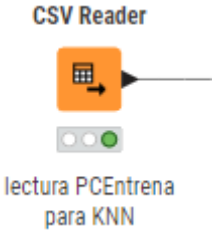
Vamos a usar el dataset EPClasifica para ver como quedarían los datos clasificados.



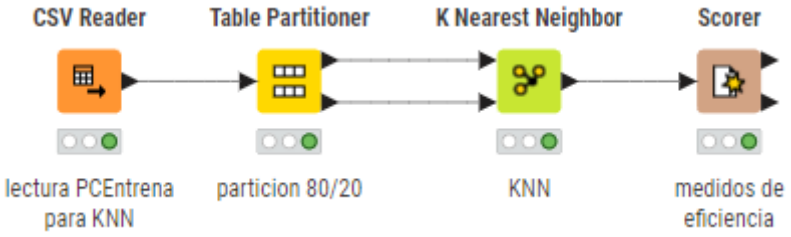
Img_part Number (Float)	Img_part2 Number (Float)	Real_part Number (Float)	Real_part2 Number (Float)	Genero Number (Integ...	Edad Number (Integ...	Fumador Number (Integ...	Prediction ... String
-320.61	-300.564	-495.26	-464.172	1	77	2	COPD
-327.78	-317.4	-473.73	-468.856	1	76	2	COPD
-325.39	-316.156	-478.52	-472.87	0	65	2	COPD
-327.78	-318.678	-507.23	-469.024	1	60	2	COPD
-323	-314.487	-473.73	-466.981	1	77	2	COPD
-320.61	-307.6	-476.12	-470.182	1	74	2	COPD
-320.61	-308.118	-471.34	-466.131	1	65	2	COPD
-315.82	-294.27	-471.34	-463.193	0	56	2	HC
-313.43	-299.055	-521.58	-421.189	1	75	2	COPD
-311.04	-297.99	-466.55	-428.95	0	65	2	COPD
-323	-314.487	-473.73	-466.981	1	45	2	HC

KNN

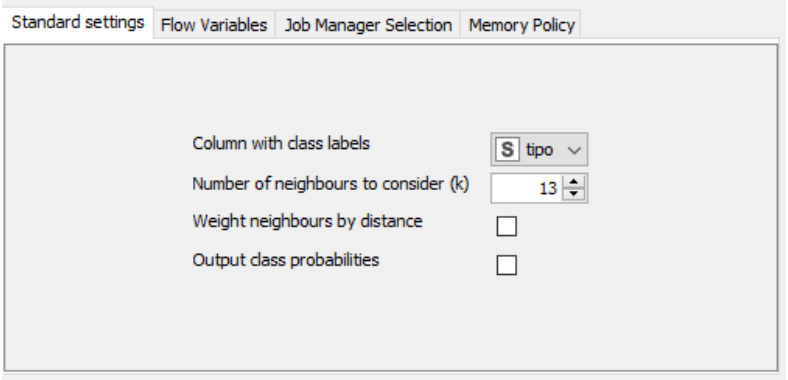
Vamos a trabajar con el dataset PCEntrena que ya hemos preprocesado.



Armamos el flujo de datos con un Partitioner de 80/20 como veníamos haciendo y una semilla de 123456, y lo conectamos al nodo K Nearest Neighbor y luego al Scorer para medir la eficiencia.



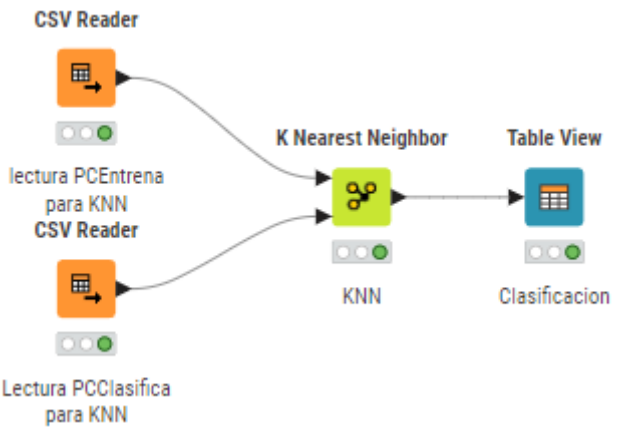
A diferencia de los otros algoritmos ,este no usa entrenador , se conecta y calcula directamente.
El K que tendríamos que usar en base a nuestro dataset de 25000 instancias será de 13, ya que la raíz cuarta es de 12.57 y redondeamos al próximo número impar.



Con esta configuración la eficiencia es de un 100%. Para confirmar que esta eficiencia del 100% es estable incluso con diferentes números de vecinos vamos a probar con otros valores de K(3/5/7/9/11).

Número K	Eficiencia
3	100%
5	100%
7	100%
9	100%
11	100%

Estos resultados sugieren que, con nuestro modelo, no hay problemas de sobreajuste cuando usamos valores de K chicos, ni de subajuste si K es muy grande.La separación casi lineal de los tipos en el espacio de características se debe a que nuestro conjunto de datos contiene atributos altamente discriminativos, como el form_factor. Esta característica facilita significativamente la clasificación correcta de cada tipo por el algoritmo, sin importar el número de vecinos considerados.
Por último pasamos el dataset de PCClasifica para observar la clasificación en base a los nuevos datos.



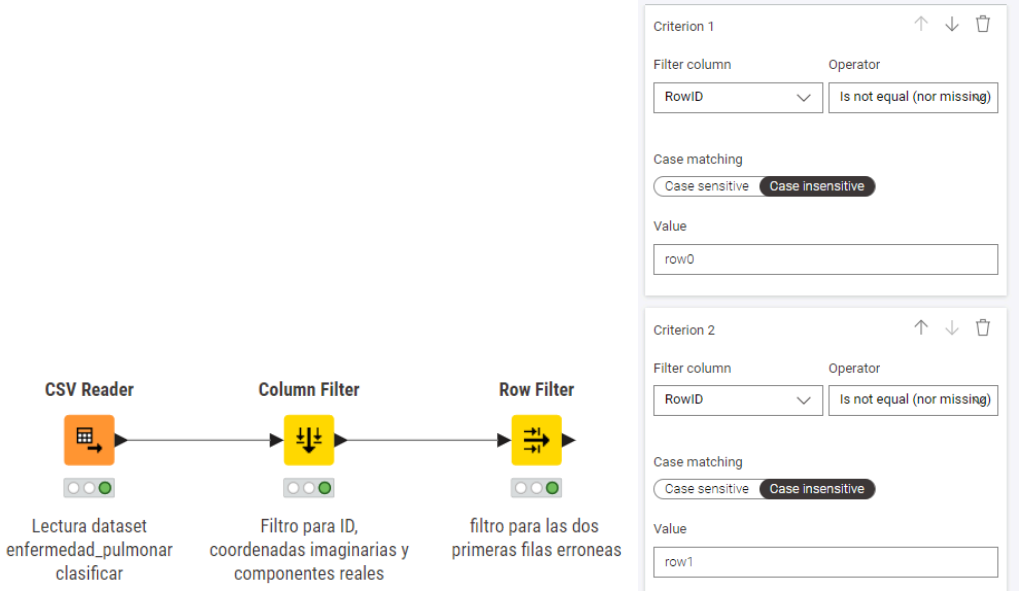
Y vemos los resultados obtenidos.

ram_gb Number (Float)	pantalla_t... Number (Float)	battery_wh Number (Float)	charger_w... Number (Float)	psu_watts Number (Float)	bluetooth Number (Float)	peso_kg Number (Float)	Form_fact... Number (Float)	AMD Number (Float)	Intel Number (Float)	Apple Number (Float)	NVIDIA Number (Float)	Wifi Number (Float)	Class [kNN] String
1.747	1.022	-1.166	-0.99	1.348	-0.784	2.313	-1.221	1.35	-1.06	-0.364	-1.11	1.863	Desktop
0.999	-0.912	0.507	0.059	-0.769	-0.784	-0.717	0.819	-0.741	0.943	-0.364	0.901	1.863	Laptop
-0.248	-0.912	0.785	0.462	-0.769	-0.103	-0.823	0.819	1.35	-1.06	-0.364	-1.11	0.87	Laptop
-0.248	1.765	-1.166	-0.99	0.078	-0.784	-0.072	-1.221	-0.741	0.943	-0.364	-1.11	-0.124	Desktop
0.75	0.576	-1.166	-0.99	1.066	0.578	0.988	-1.221	1.35	-1.06	-0.364	0.901	-0.124	Desktop
-0.248	-0.674	1.343	1.915	-0.769	-0.103	-0.735	0.819	-0.741	0.943	-0.364	0.901	-0.124	Laptop
0.75	-0.674	0.507	1.915	-0.769	0.578	-0.735	0.819	-0.741	0.943	-0.364	0.901	-0.124	Laptop
-0.746	1.022	-1.166	-0.99	1.348	-0.103	0.723	-1.221	1.35	-1.06	-0.364	0.901	-0.124	Desktop
-0.248	-0.912	1.343	1.915	-0.769	0.578	-0.823	0.819	-0.741	0.943	-0.364	0.901	-0.124	Laptop
2.744	0.576	-1.166	-0.99	0.078	1.259	0.988	-1.221	1.35	-1.06	-0.364	0.901	0.87	Desktop
1.747	2.063	-1.166	-0.99	1.066	0.578	-0.072	-1.221	-0.741	-1.06	2.751	-1.11	0.87	Desktop

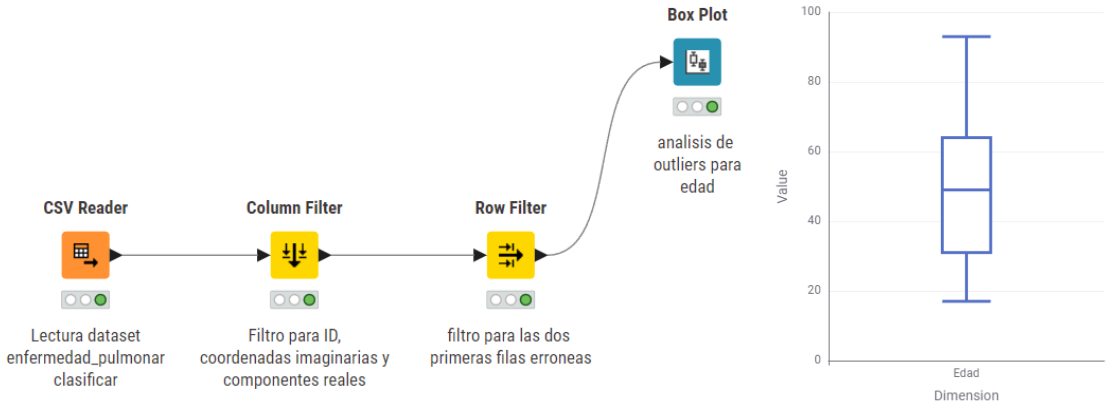
K-Means

Trabajaremos con el conjunto de datos enfermedadpulmonar_clasifica.csv ya que para este algoritmos no necesitamos la etiqueta de diagnóstico. Dado que este dataset presenta un 77% de valores faltantes, optaremos por filtrar esos atributos. Esta decisión se basa en que reemplazar un porcentaje tan alto con la media o la mediana no resultaría representativo, comprometiendo la integridad de los datos. La eliminación de estas columnas constituye una técnica de reducción de dimensionalidad que asegura que el conjunto de datos resultante esté 100% completo y sea confiable. Además que ya en árboles de decisión comprobamos que estos atributos tienen un bajo peso a la hora de clasificar.

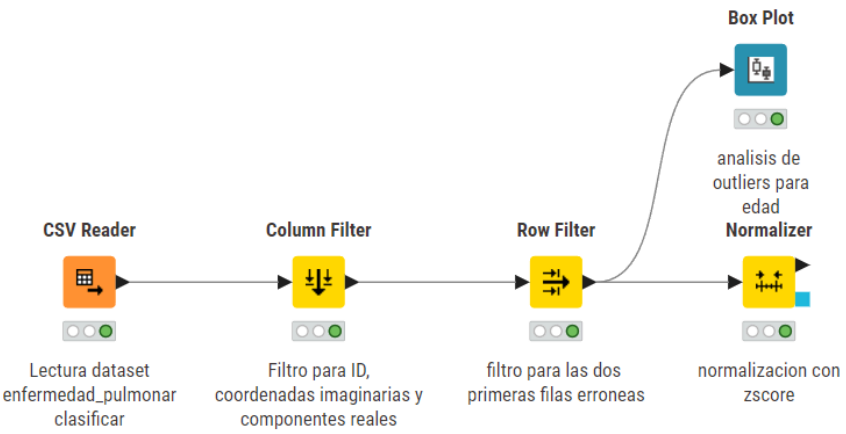
Además en este dataset las dos primeras filas poseen datos faltantes en todos los atributos por lo que también las filtraremos.



Como este algoritmo es sensible a outliers, ya que pueden distorsionar significativamente la posición de los centroides, vamos a analizar si hay presencia de estos. La única variable que podría contener outliers es la edad, ya que género y fumador solo tienen dos y tres parámetros respectivamente.



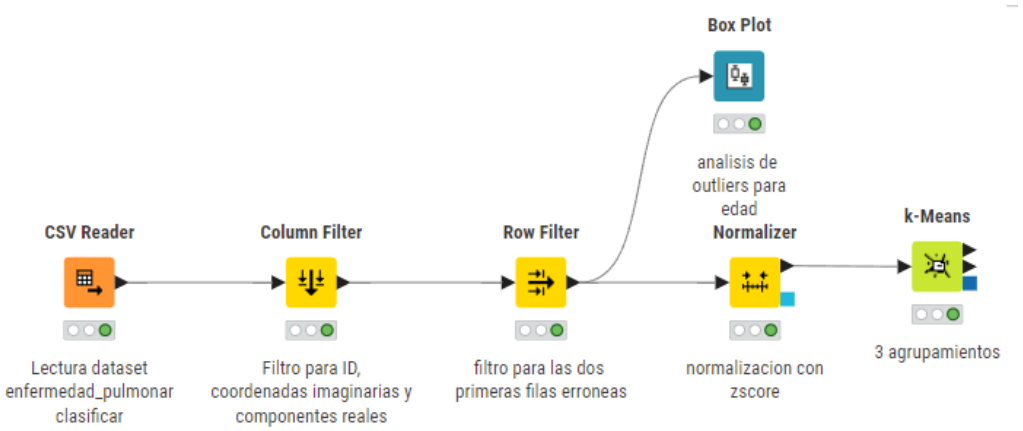
Podemos ver a través del boxplot que no tenemos valores atípicos.
Para asegurar que todas las características contribuyan equitativamente a la distancia, permitiendo que K-Means descubra la estructura del cluster basándose en la combinación de todos los datos vamos a normalizar con Z-Score para centrar los datos.



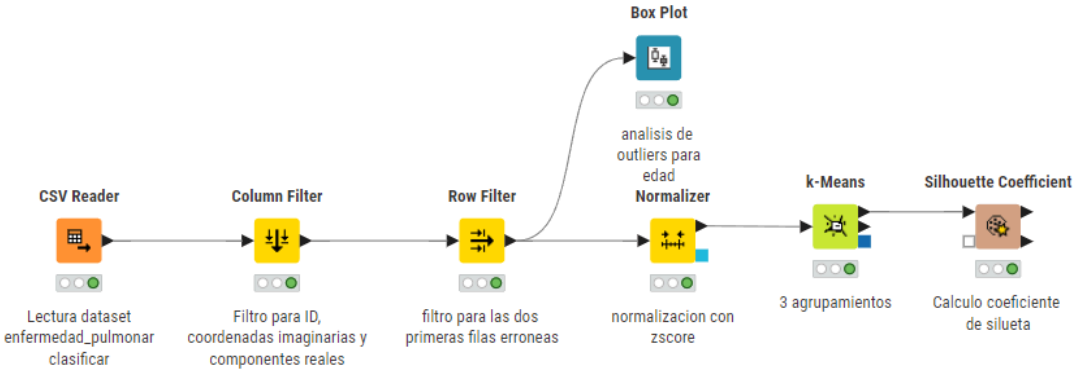
Y como ya nuestros datos son todos numéricos no tenemos que hacer más transformaciones.

Name	Type	# Missing val...
Genero	Number (Float)	0
Edad	Number (Float)	0
Fumador	Number (Float)	0

A continuación, procederemos a conectar el nodo de k-Means. Lo configuraremos inicialmente para generar 3 agrupamientos con una semilla de 123456, y posteriormente repetiremos la configuración con 4 agrupamientos, manteniendo la misma semilla.



Para calcular el coeficiente de silueta conectamos el nodo Silhouette Coefficient.

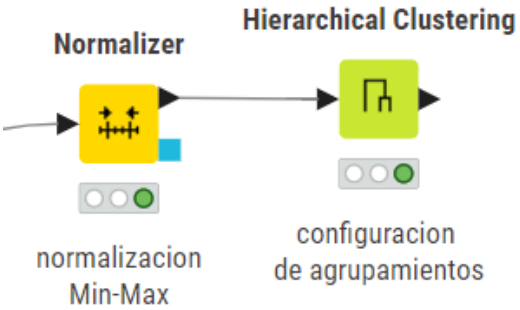


Agrupamientos	Coeficiente de silueta
3	0.404
4	0.461

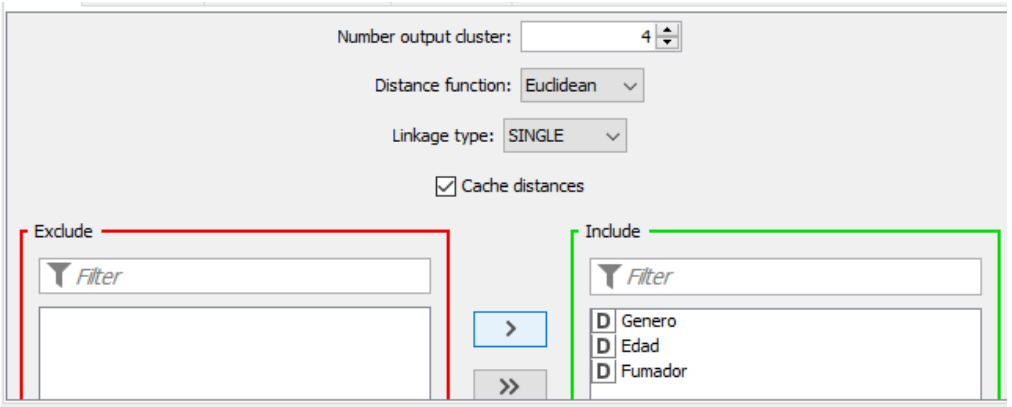
Dado que un coeficiente de silueta más cercano a uno indica una mejor agrupación, podemos concluir que la opción más óptima es la de 4 clústeres, si bien la diferencia no es significativa.

Clustering Jerárquico

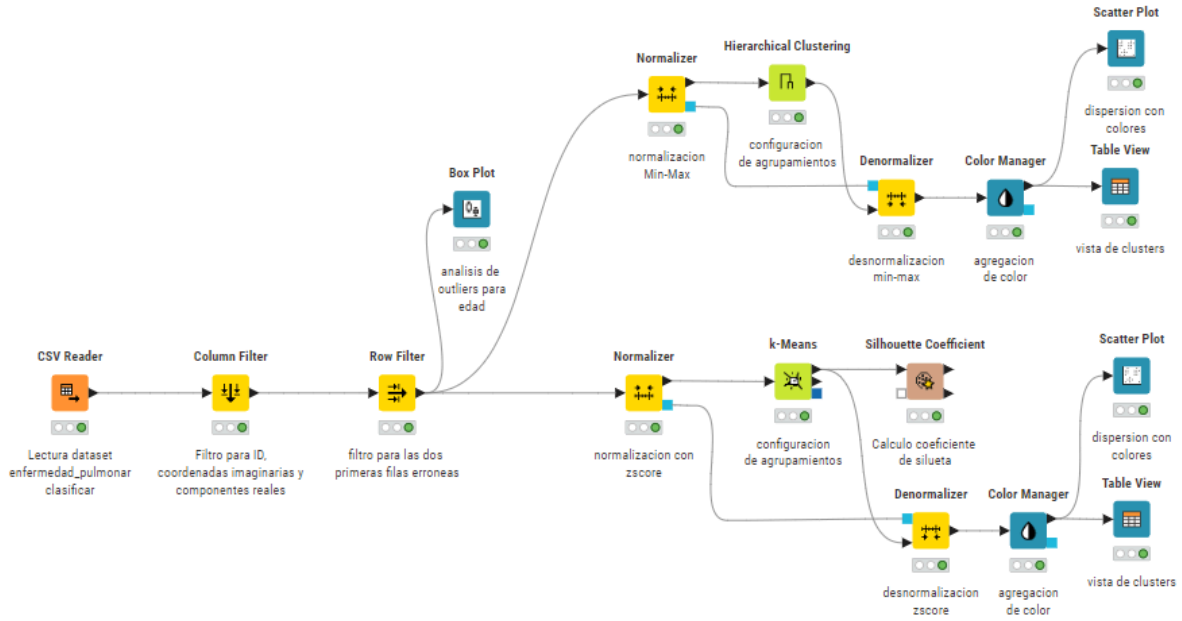
Para realizar este punto vamos a usar el mismo preprocesamiento que hicimos para el punto de k-Means ya que en ambos tenemos que normalizar y trabajar, pero la normalización la realizaremos con Min-Max entre 0 y 1. Luego conectamos el Normalize al nodo Hierarchical Clustering.



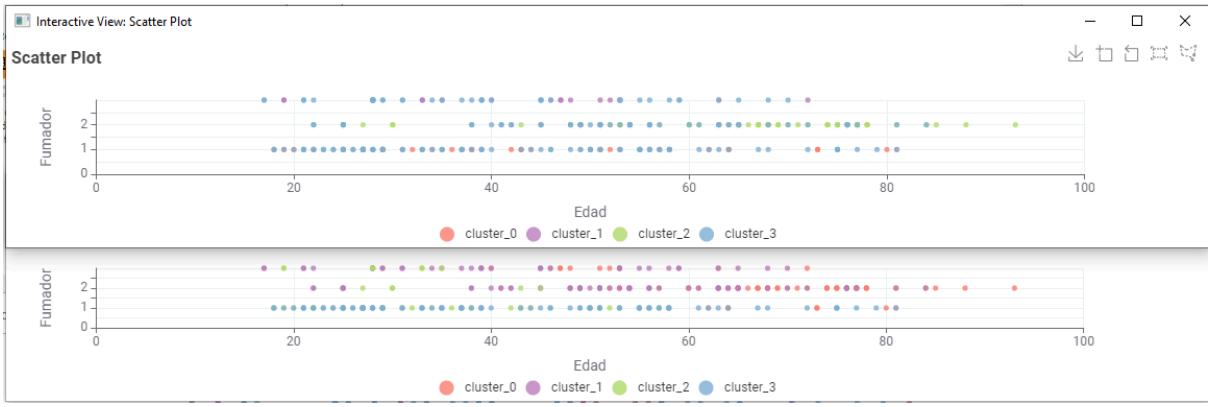
Configuramos el nodo con 4 agrupamientos ,distancia Euclidean y con vinculación simple.



Desnormalizamos para que los gráficos sean más claros y les ponemos color.



Para el diagrama de dispersión, seleccionaremos las variables de edad y si el individuo es fumador. Estas son las más significativas, ya que la variable de género no aporta la dispersión necesaria, lo que no nos permite maximizar el valor informativo del gráfico.

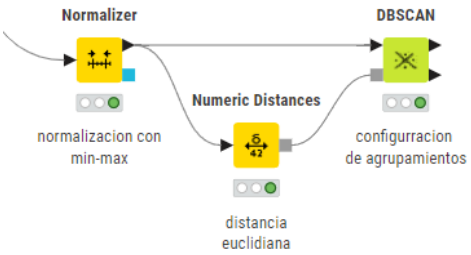


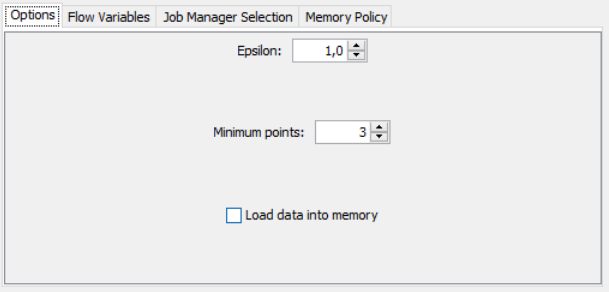
El gráfico de arriba ilustra la dispersión del clustering jerárquico, mientras que el de abajo representa la dispersión del método k-means.

Se observa una diferencia en la distribución de los datos entre ambos métodos: el algoritmo k-means concentra una mayor cantidad de datos en el cluster 1, mientras que el clustering jerárquico tiende a clasificar más datos en el cluster 3.El algoritmo K-Means es el que mejor funciona con estos datos. La diferencia principal está en cómo reparte la población: K-Means busca grupos más o menos iguales, mientras que el método jerárquico los arma basándose en el historial. Los gráficos confirman que K-Means es la mejor opción, porque sus grupos están más juntos y mejor separados entre sí.

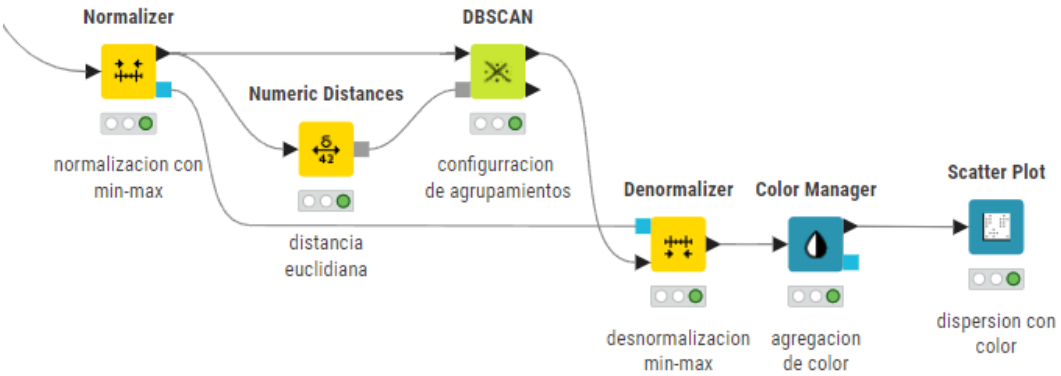
DBScan

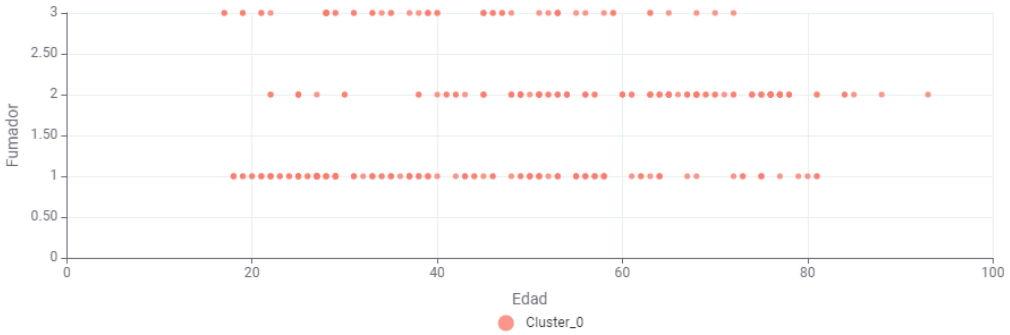
Vamos a mantener el preprocesamiento anterior pero vamos a normalizar con Min-Max entre las características 0 y 1. Vamos a realizar la agrupación con la configuración predeterminada.



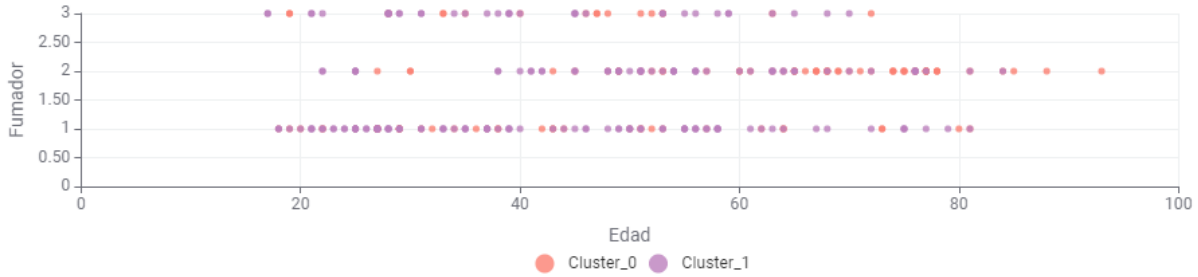


Visualizamos los clústeres producidos por el DBScan con un diagrama de dispersión.





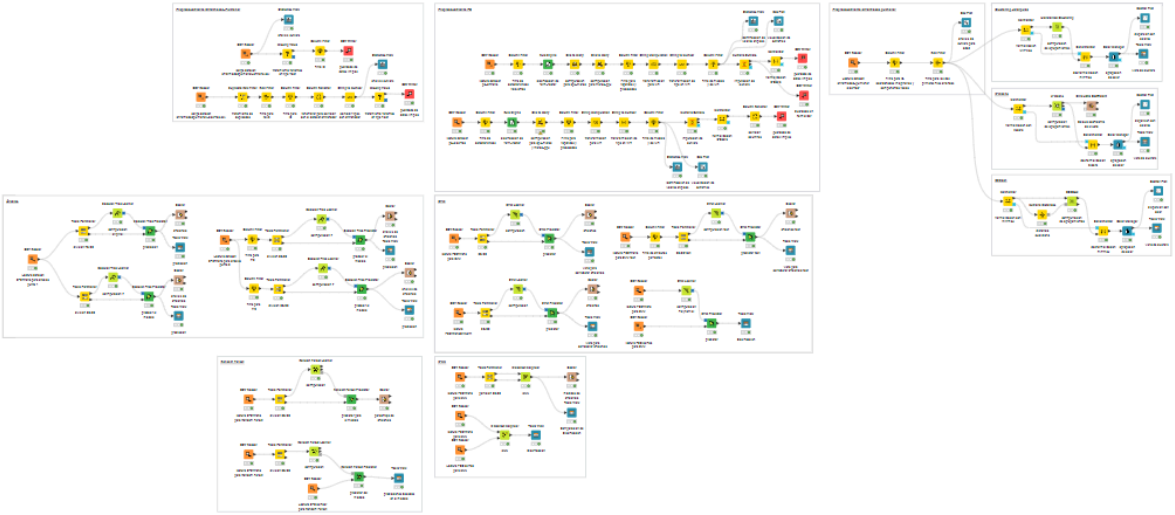
Podemos ver que con la configuración predeterminada el algoritmo nos crea un único cluster. Vamos a cambiar el número de Epsilon a 0,5.



Con esto podemos ver que ahora sí encontró una división entre los datos y creó 2 clusters.
Ahora vamos a aumentar el número de puntos mínimos a 5.

Con esta configuración observamos que igualmente nos dividió los datos en 2 clusters y la distribución es bastante parecida a la anterior, no hay cambio significativos entre variar el número de puntos mínimos. El ajuste de epsilon a 0,5 fue el punto clave que obligó a DBSCAN a pasar de ver todo como un solo grupo (un cluster gigante) a revelar la verdadera estructura de 2 clusters.

Conclusiones obtenidas de las actividades de agrupamiento

Después de comparar las tres técnicas de clustering, podemos decir que K-Means demostró ser la mejor opción y la más sólida. Esto se debe a que K-Means agrupa los datos basándose en la similitud. Esta forma de trabajar encajó perfectamente con las variables que elegimos (edad, género y fumador), las cuales definen categorías de forma natural. En cambio, el método Jerárquico generó grupos mezclados porque es demasiado rígido. Por su parte, DBScan fue muy sensible a la densidad, y eso hizo que los resultados variarán muchísimo con solo cambiar un poco sus parámetros.
En síntesis, K-Means fue el algoritmo que logró captar de forma óptima la estructura oculta de los datos, resultando en grupos muy compactos, consistentes y muy fáciles de distinguir.

Flujo final de datos